

Retrieval - Augmented Generation (RAG)

① Introduction to RAG

- What is RAG
- Why RAG Exists
- RAG Pipeline

② AI & NLP

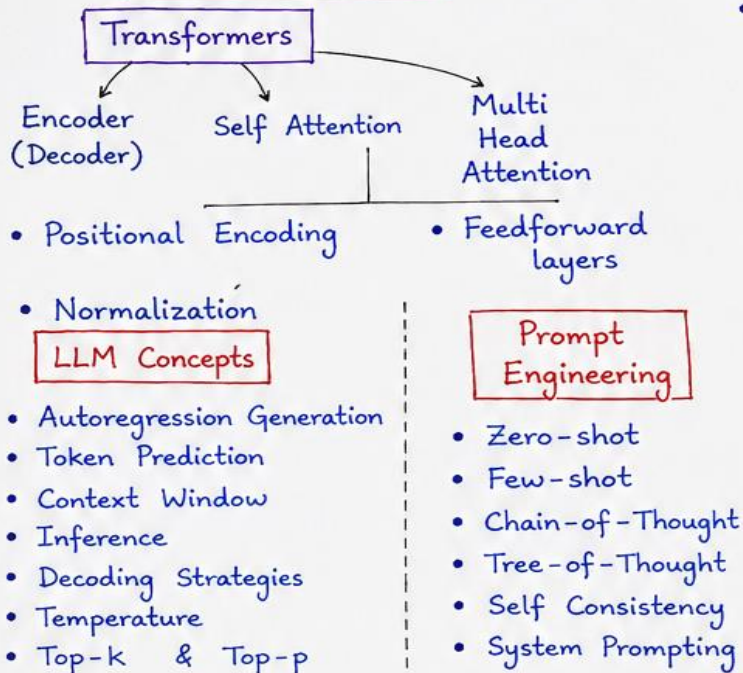
Fundamentals

- Basics, Python Libraries
- NLP Basics

Tokenization

- Stemming
- Lemmatization
- TF-IDF
- BM25
- n-gram
- Stopwords

③ ^{★★} Transformers & LLM Foundations



④ Embeddings & Vectors

- What are Embeddings
- Vectors Math

- Important Topics**
- Vectors
 - Dot product
 - Cosine Similarity
 - Matrices
 - Normalization
 - Euclidean Distance

Embedding

- Dense Embedding
- Sparse Embedding
- Multi-vector Embeddings

⑤ ANN Algorithms

HNSW

- Graph Construction
- EF Search
- EF Construction
- Graph Traversal
- Memory Tradeoffs

IVF

- Clustering
- Centroid Based
- Direct
- Probe

PQ

- Vector Compression
- Quantization
- Sub Vector Encoding

ScaNN

Google Anti Retrieval

Concepts to Master

- Meta Data
- Collections / Indexes
- Namespace
- Persistence
- Sharding
- Replication
- Distributed Retrieval
- Hybrid Search

⑥ Vector Databases

- | | | | |
|------------|------------|-----------|------------|
| • ChromaDB | • Pinecone | • Milvus | • NeonDB |
| • FAISS | • Qdrant | • Vespa | • PGVector |
| | • Weaviate | • LanceDB | |

⑦ RAG Architecture

Chunking Techniques

- Fixed Chunking
- Recursive Chunking
- Semantic Chunking
- Token-based Chunking
- Sliding Window Chunking
- Parent-Child Chunking

Re-ranking

Adaptive Chunking

Dynamic Chunk Routing

AI Memory System

- Semantic Memory
- Episodic Memory
- Working Memory
- Long-term Memory

⑧ Embedding Models

- MiniLM
- mpnet
- BGE
- E5
- Instructor XL
- OpenAI Embeddings
- Voyage AI Embeddings
- Cohere Embeddings

Research Topic

- Domain-specific Embeddings
- Embedding fine-tuning
- Multilingual Embeddings
- Retrieval-aware Embeddings

9 Retrieval Techniques → Advanced Retrieval

Sparse Retrieval

- BM25
- SPLADE
- TF-IDF

Hybrid Retrieval

- Dense retrieval
- Sparse retrieval

- multi-query retrieval
- recursive retrieval
- adaptive retrieval
- semantic caching

10 → LongChain

- Chains
- Prompts
- Retrievers
- Memory
- Tools
- Agents
- Retrieval QA
- Conversational Retrieval Chain
- Router Chains
- Agent Executors

Advanced Topic

- LangGraph
- Tool Calling
- Memory System
- async pipelines

- RAG Variants
- Context selection
- Compression
- Central mode

12 Advanced RAG

- retrieval
- reranking
- query rewriting
- compression
- evaluation
- fusion (RRF)
- candidate generation
- semantic chunking

11 Vector Indexes

- Query Engine
- Recursive Retrieval
- Graph Indexes
- Tree Indexes
- Router Retrieval
- composable indexes
- Sub-question engines
- Knowledge graph retrieval

14 Types of RAG

- Adaptive RAG
- Self-reflective RAG
- Corrective RAG
- Active RAG
- Dense retrieval
- Sparse retrieval

15 AI Memory System

- Semantic Memory
- Episodic Memory
- Working Memory
- Long-term Memory

17 Research Topic

- Domain-specific Embeddings
- Embedding fine-tuning
- Multilingual Embeddings
- Retrieval-aware Embeddings

18 Embedding Models

- MiniLM
- mpnet
- BGE
- E5
- Instructor XL
- OpenAI Embeddings
- Voyage AI Embeddings
- Cohere Embeddings

19 Important Topic

- Domain-specific Embeddings
- Collections / Indexes
- Namespace
- Persistence
- Sharding
- Replication
- Distributed Retrieval
- Hybrid Search

13 Hybrid search → SPLADE retrieval



- BM25 fusion
- reciprocal rank fusion
- weighted hybrid retrieval

16 Hypothetical search → SPLADE retrieval



- BM25 fusion
- reciprocal rank fusion
- weighted hybrid retrieval

RAG

= Retrieve relevant information
from Documents

(use context to LLM)

Generate accurate answers

Pipeline

RAG = information retrieval + language model

Goal: Augmentation
of LLM knowledge
by retrieving context
from external sources
(knowledge base /
documents / DB)
→ Improve model
performance.

Documents → chunking → Embeddings → Vector Database

- LangChain / LlamaIndex
- Chroma DB
- FAISS
- Sentence-Transformers

LLM Limitations

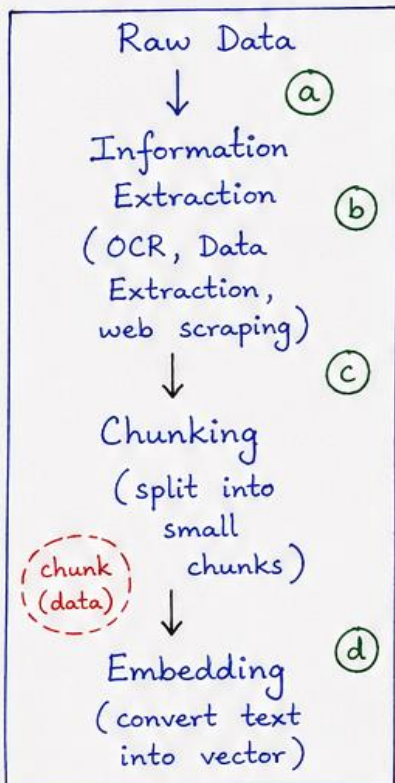
- Trained only on old data
- Cannot access private docs
- Inaccurate
- Limited knowledge updates

Similarity Search

→ Retrieved context

→ LLM response

Workflow



Text Embedding
Conversion

Comparison
&
Selection

Semantic
matching

Vector
Database

① Embedding
query

Retrieval
(Data Fetch)

③ Relevant
data
(based upon
top similar
vectors)

④ LLM

⑤ Response

Key Takeaway ★

RAG combines the power of retrieval (search) from relevant data sources with the generation ability of LLMs to deliver accurate, up-to-date and context-aware answers.

RAG vs Fine-tune

RAG →

- Retrieves external information
- adapts to the latest insights using real world data.

Fine-tune →

- Retrains or adapts the model weights using new training data.

★ Exam Tip

Always remember
RAG = Retrieve
→ Augment
→ Generate

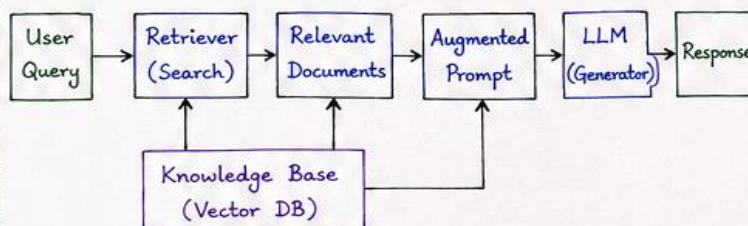
★ RAG (Retrieval-Augmented Generation) ★

RAG combines the power of information retrieval and LLM generation to produce accurate, up-to-date and context-aware answers.

1. What is RAG ?

- RAG is a framework that augments LLMs with external knowledge.
- Instead of relying only on the model's internal knowledge, RAG fetches relevant information from external sources and uses it to generate the final response.

RAG Architecture (High Level)



★ Exam Tip

RAG = Retrieve (Search relevant data) + Augment (add to prompt) + Generate (LLM produces answer)

2. Why use RAG ?

- Reduces hallucinations
- Provides up-to-date information
- Domain specific knowledge
- Improves transparency (with citations)
- Cost effective (no need for fine-tuning)
- Easy to update (just update KB)

3. Key Components of RAG

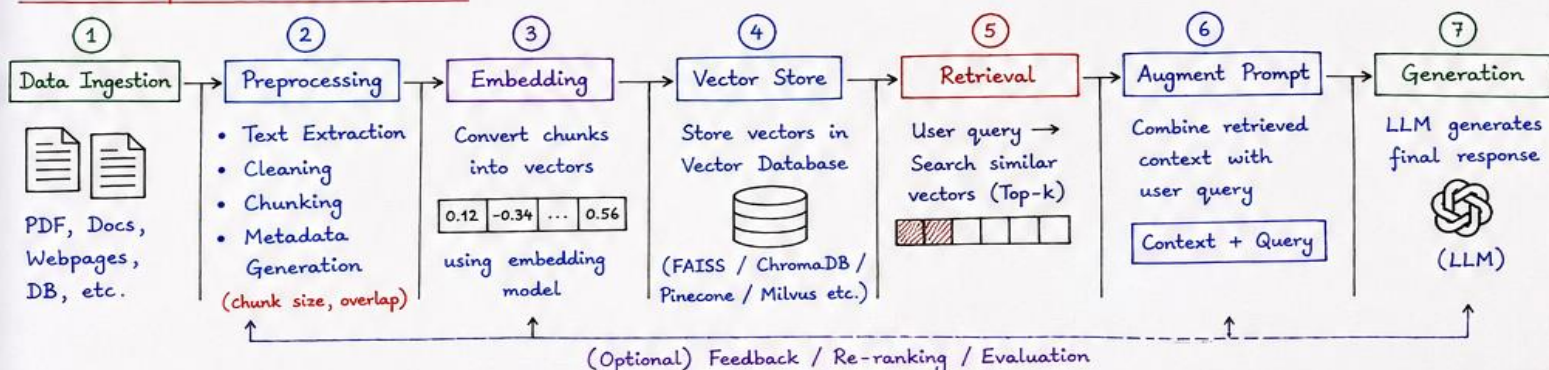
- Data Sources → Documents, PDFs, Websites, DBs
- Retriever → Search relevant information
- Vector Store → Stores embeddings of documents
- LLM → Generates the final answer
- Prompt Template → Combines context + query
- Output Parser → Formats the final output

Remember

Better Retrieval
= Better Context
= Better Answer



4. RAG Pipeline - Detailed Flow



5. Retrieval Techniques

- Dense Retrieval - Using embeddings (semantic search)
- Sparse Retrieval - Using keywords (BM25)
- Hybrid Retrieval - Combination of dense + sparse
- Re-ranking - Reorder results using cross-encoder

6. Retriever Types

- Vector Similarity Search (Top-k)
- MMR (Maximal Marginal Relevance)
- Semantic Search
- Metadata Filtering
- Self Query Retriever (LangChain)
- Parent Document Retriever

7. Vector Databases

- ChromaDB
- Pinecone
- FAISS
- Milvus
- Weaviate
- Qdrant
- Vespa
- LanceDB
- PGVector
- NeonDB

8. LLM Role in RAG

- Understand the query
- Use retrieved context to answer
- Provide citations (source aware)
- Follow instructions from prompt
- Avoid hallucinations

Exam Tip

LLM is not fine-tuned. It just uses retrieved context to generate better answers.

9. Prompt Engineering in RAG

You are a helpful assistant.
Use the following context to answer the question.
If the answer is not in the context, say you don't know.
Context:
{context}
Question: {query}
Answer:

System / Instruction

Retrieved Context

User Query

10. Post-Processing

- Re-ranking (Cross Encoder)
- Context Compression / Summarization
- Answer Filtering
- Deduplication
- Hallucination Detection

11. Evaluation Metrics

- Faithfulness
- Context Precision / Recall
- Answer Relevancy
- Exact Match (EM)
- F1 Score
- Rouge / BLEU (for generation)

12. Advantages of RAG

- ✓ Up-to-date knowledge
- ✓ Reduces hallucinations
- ✓ Domain specific
- ✓ Transparent (with sources)
- ✓ Cost effective
- ✓ Easy to maintain

13. Limitations of RAG

- ✗ Depends on quality of data
- ✗ Retriever may miss relevant info
- ✗ Long context increases cost
- ✗ Complex pipeline
- ✗ Not 100% accurate

Interview Question

- Q. What is RAG and how does it improve the performance of LLMs?
- Q. What are the components of RAG pipeline?

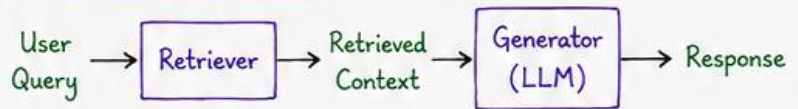
★ **Key Takeaway:** RAG enhances LLMs by grounding them in external knowledge.
Better retrieval → better context → better generation.

RAG Architectures

① What is RAG Architecture?

- Combines Retrieval + Generation
- Grounded, Accurate, Up-to-date
- Reduces Hallucinations
- Modular & Extensible

② Core Components



- Retriever : Finds relevant information
- Generator : Generates answer using context

③ RAG Architecture Patterns

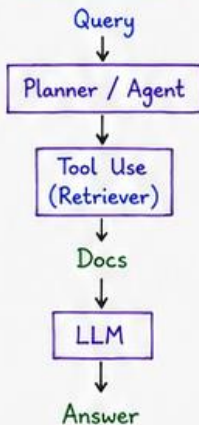
Naive RAG



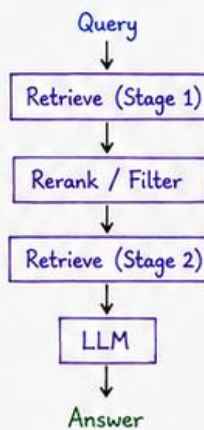
Advanced RAG



Agentic RAG



Multi-Stage RAG



④ Retrieval Strategies

- Dense Retrieval (Embeddings)
- Sparse Retrieval (BM25)
- Hybrid Retrieval (Dense + Sparse)
- Multi-vector Retrieval
- Contextual Compression
- Metadata Filtering
- Graph-based Retrieval

⑤ Indexing & Storage

- Chunking Strategies (Fixed / Recursive / Semantic)
- Vector Databases (FAISS, Pinecone, Milvus, Weaviate)
- Keyword Indexes (Elasticsearch, OpenSearch)
- Graph Databases (Neo4j, TigerGraph)

⑥ RAG Architecture Variants

Self-RAG

- LLM decides what to retrieve
- Reflects on own answers
- Iterative retrieval & generation

Corrective RAG

- LLM checks faithfulness
- Retrieves more if needed
- Corrects final answer

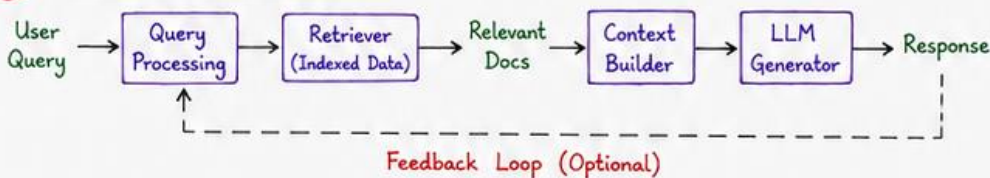
Adaptive RAG

- Chooses best retrieval strategy based on query
- Dynamic k, filters, or routing

Routing RAG

- Routes query to right data source or retriever
- Domain / intent based routing

⑦ Data Flow in RAG



⑧ Context Construction

- Stuffing (Concatenate all)
- Map-Reduce
- Refine (Iterative)
- Tree-based Summarization
- Sliding Window
- Context Compression

⑨ Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Relevance (Faithfulness)
- Noise Robustness
- Hallucination Rate
- Answer Correctness (Exact / F1)
- Latency & Throughput

⑩ Challenges

- Irrelevant Retrieval
- Outdated Information
- Large Context Window Cost
- Hallucinations
- Chunking Quality
- Scaling & Index Updates

⑪ Optimization Techniques

- Better Chunking
- Query Expansion / Reformulation
- Reranking Models
- Embedding Fine-tuning
- Caching Frequent Results
- Adaptive Retrieval Depth

⑫ RAG Architectures at a Glance



⑬ Best Practices

- Use Hybrid Retrieval
- Rerank before generation
- Keep context concise but relevant
- Continuously evaluate

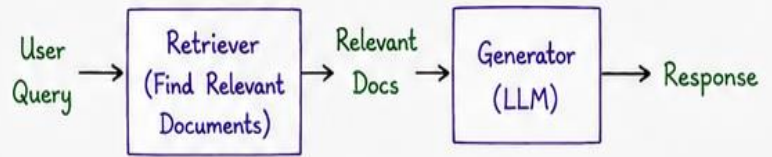
Traditional RAG

(Retrieval-Augmented Generation)

① What is Traditional RAG?

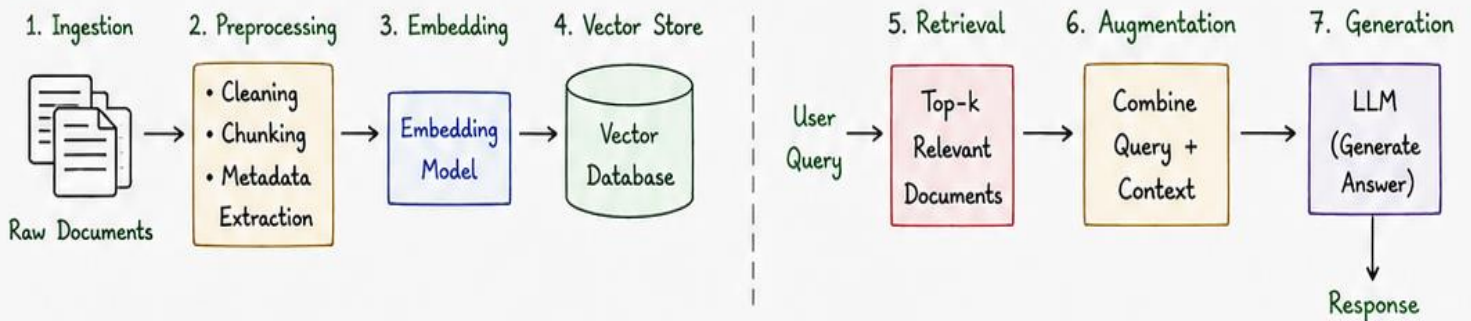
- Combines LLMs with external knowledge retrieval.
- Retrieves relevant documents from a knowledge base and augments the prompt.
- Improves accuracy and reduces hallucinations.
- Easy to implement and widely used.

② Core Components

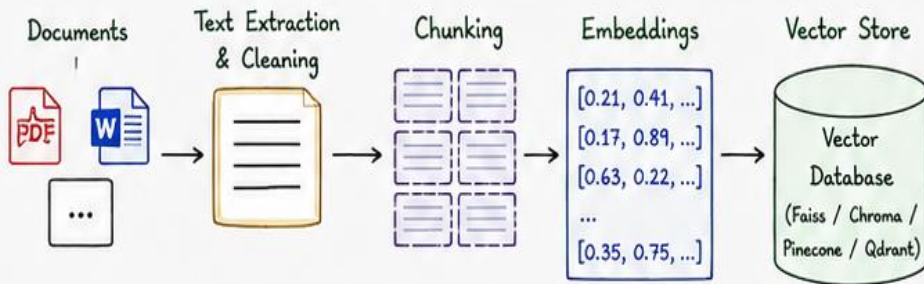


- Retriever : Searches and returns relevant context
- Generator : Uses the context + query to generate answer

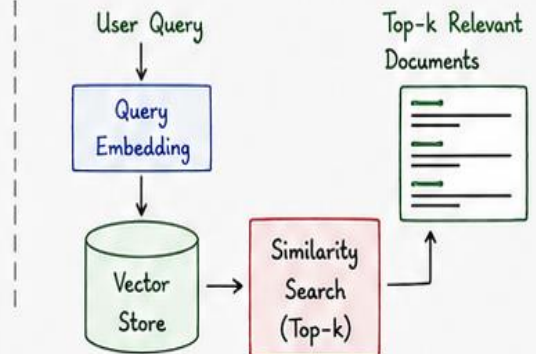
③ Traditional RAG Pipeline



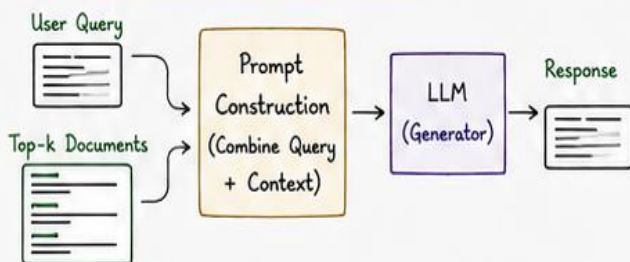
④ Ingestion & Indexing (Offline)



⑤ Retrieval (Online)



⑥ Augmentation & Generation



⑦ Advantages

- Simple and easy to implement
- Uses up-to-date external knowledge
- Reduces hallucinations
- Cost-effective with open-source models
- Scalable with vector databases

⑧ Limitations

- Depends on retrieval quality
- May miss information (recall issue)
- Large context window can be costly
- Needs good chunking & embeddings
- Not good for complex reasoning

⑨ Common Vector Databases

- | | |
|------------|------------|
| • FAISS | • Weaviate |
| • Chroma | • Qdrant |
| • Pinecone | • Milvus |

⑩ Common Embedding Models

- | | |
|--|--------------------------|
| • OpenAI (text-embedding-3-small/large) | • E5 (intfloat/e5-large) |
| • sentence-transformers (all-MiniLM-L6-v2) | • Instructor / GTE |
| • BGE (BAAI/bge-large-en) | • Cohere Embed |

⑪ Best Practices

- Use meaningful chunking
- Add metadata & filters
- Retrieve more (Top-k = 5~20)
- Re-rank if possible
- Continuously evaluate & improve

Naive RAG

What is Naive RAG?

- Simplest form of Retrieval-Augmented Generation
- Retrieves top-k relevant documents and passes them to the LLM
- Easy to build and understand
- Works well for straightforward Q&A tasks

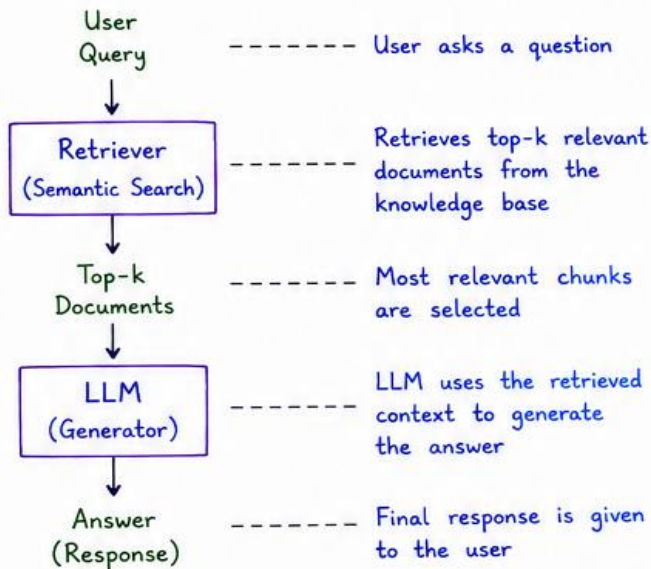
Overview (Flow)



Key Idea

Find relevant context → Give to LLM → Generate answer

Architecture (Flow Diagram)



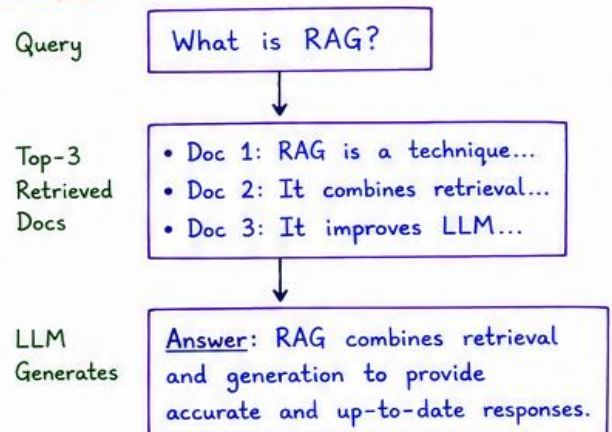
Components

- **User Query**
The question or input provided by the user.
- **Retriever**
Finds relevant documents/chunks using similarity search (e.g., Vector DB).
- **Knowledge Base**
Collection of documents split into chunks and stored as embeddings.
- **Generator (LLM)**
Uses retrieved context to generate accurate answer.

How it Works (Step-by-Step)

- ① User provides a query.
- ② Retriever searches the knowledge base.
- ③ Top-k relevant chunks are retrieved.
- ④ Retrieved chunks are given to the LLM along with the query.
- ⑤ LLM generates the answer using the context.
- ⑥ Response is returned to the user.

Example



Pros

- ✓ Simple to implement
- ✓ Fast and efficient
- ✓ Works well for basic Q&A
- ✓ Easy to maintain

Cons

- ✗ May miss relevant info if not in top-k
- ✗ No re-ranking or filtering
- ✗ May include irrelevant or noisy context
- ✗ Limited for complex queries

Use Cases

- FAQ systems
- Customer support bots
- Document search with Q&A
- Internal knowledge bases
- Basic research assistants

Notes

- Naive RAG is the foundation for advanced RAG architectures.
- Can be improved with re-ranking, filtering, and multi-stage retrieval.
- Works best when the knowledge base is high-quality and well-chunked.

Advanced RAG

(Beyond Traditional RAG)

What is Advanced RAG?

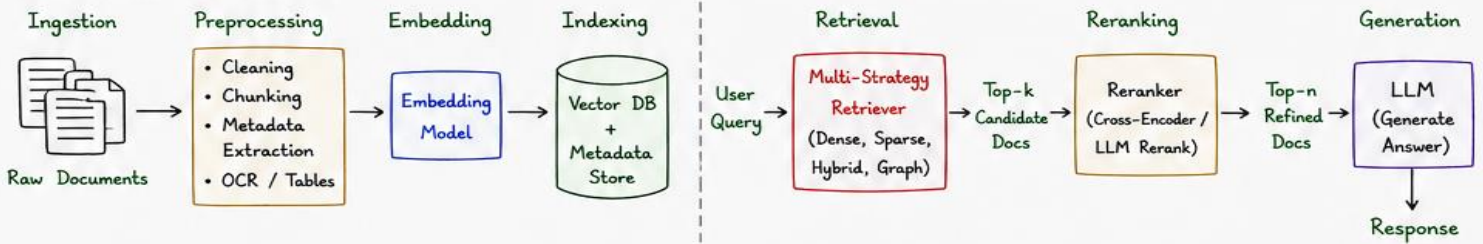
- Extends traditional RAG with smarter retrieval, reasoning & adaptation.
- Handles complex queries with better accuracy and relevance.
- Uses multiple techniques to retrieve, rerank, refine and reason.
- Built for scalability, reliability and real-world production.

Core Components



- Advanced Retriever : Uses multiple retrieval strategies.
- Reranker : Reorders results for higher relevance.
- Generator : Generates accurate answer using refined context.

Advanced RAG Pipeline



Retrieval Strategies (Examples)

Dense Retrieval	Sparse Retrieval	Hybrid Retrieval	Graph Retrieval	Multi-vector Retrieval
• Vector embeddings similarity search • Captures semantic meaning	• BM25 / TF-IDF based search • Good for exact term matching	• Combines Dense + Sparse • Better recall and precision	• Uses knowledge graphs • Captures relationships	• Multiple embeddings per document • Improves recall for diverse queries

Reranking Techniques

- Cross-Encoder Models (e.g., bge-reranker)
- LLM-based Reranking
- Rank by relevance score
- Keep top-n most relevant documents

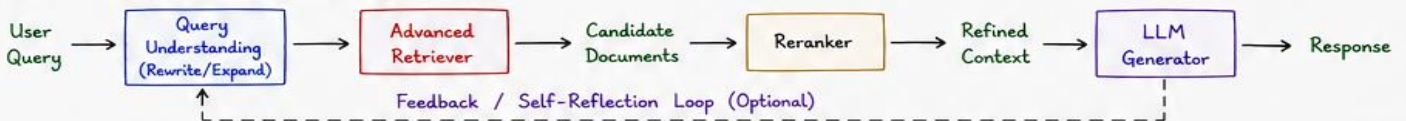
Advanced RAG Variants

Self-RAG	Agentic RAG	Adaptive RAG	Routing RAG	Multi-Stage RAG
• LLM decides what to retrieve • Reflects on its own generation • Iterative retrieve & refine	• Agent plans steps • Uses tools & retrievers • Dynamic strategy based on task	• Adapts retrieval strategy • Based on query type / domain / complexity	• Routes query to the best data source • Domain / intent based routing	• Retrieve in multiple stages • Broad → Narrow → Focused • Improves precision

Context Enhancement

- Query Expansion
- Hypothetical Document Generation (HyDE)
- Context Compression
- De-duplication
- Summarization
- Metadata Filtering

Data Flow in Advanced RAG



Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Relevance (Faithfulness)
- Noise Robustness
- Hallucination Rate
- Answer Correctness (Exact / F1)
- Latency & Throughput

Challenges

- Irrelevant Retrieval
- Outdated Information
- Large Context Window Cost
- Hallucinations
- Complex Query Understanding
- Index Updates & Maintenance

Optimization Techniques

- Better Chunking (Semantic / Adaptive)
- Reranking Models
- Caching Frequent Results
- Query Reformulation
- Parallel Retrieval
- Adaptive Retrieval Depth

Common Vector Databases

FAISS	Milvus	Redis Vector
Pinecone	Qdrant	Elastic Vector
Weaviate	Chroma	pgvector

Common Embedding Models

- OpenAI (text-embedding-3)
- BGE (BAAI/bge-large-en)
- Instructor / GTE
- E5 (intfloat/e5-large)
- Cohere Embed
- Voyage AI Embeddings

Best Practices

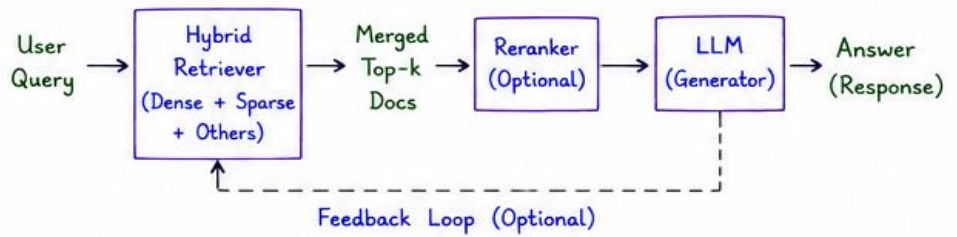
- Use Hybrid Retrieval
- Rerank before generation
- Keep context concise but relevant
- Add metadata & filters
- Continuously evaluate & improve
- Monitor & update indexes regularly

Hybrid RAG

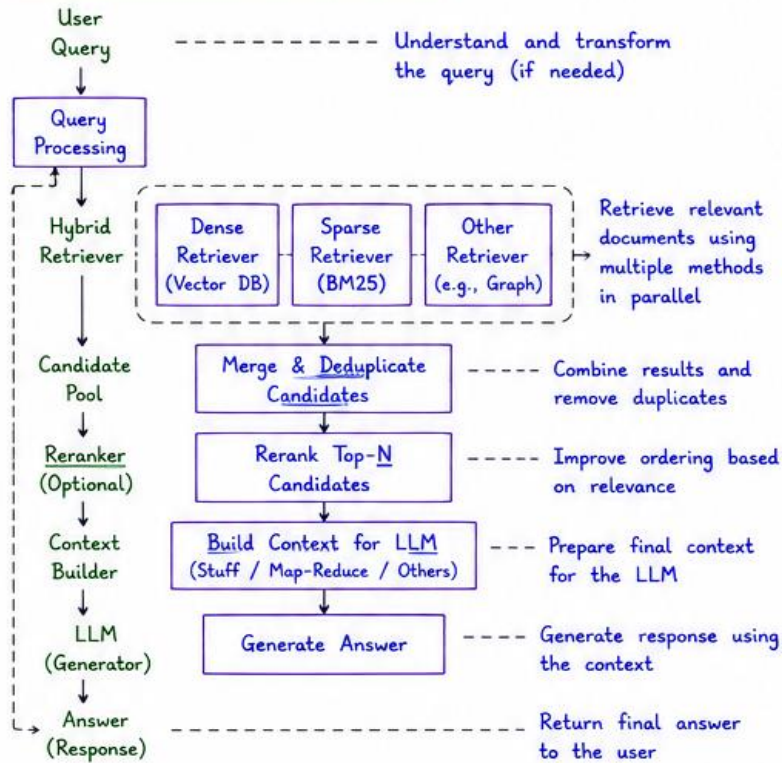
What is Hybrid RAG?

- Combines multiple retrieval strategies (e.g., Dense + Sparse + Others)
- Leverages strengths of each method
- Improves recall, relevance & robustness
- Delivers higher quality context to the LLM

Overview (Flow)



Architecture (Flow Diagram)



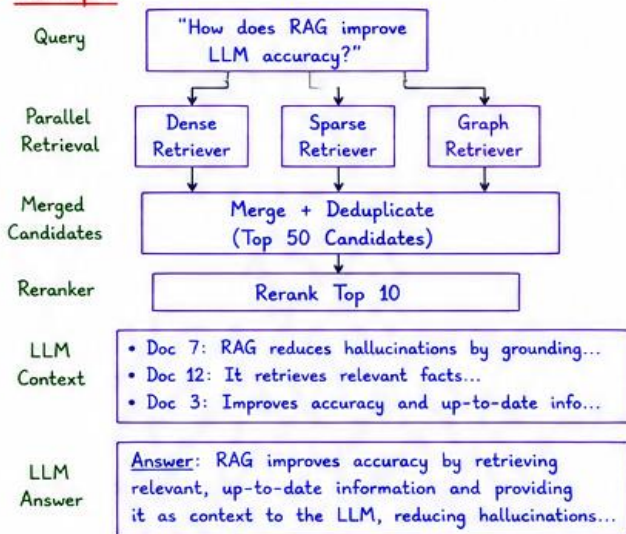
Retrieval Strategies Used

- **Dense Retrieval:** Semantic search using embeddings (Vector DBs like FAISS, Pinecone, Milvus)
- **Sparse Retrieval:** Keyword based search (BM25, TF-IDF, Elasticsearch)
- **Other Retrievals:** Graph-based, SQL-based, Metadata filters, Hybrid filters etc.

How it Works (Step-by-Step)

- ① User provides a query.
- ② Query is processed / rewritten (optional).
- ③ Multiple retrievers search in parallel.
- ④ Results are merged & duplicates are removed.
- ⑤ Candidates are reranked for better relevance.
- ⑥ Top-k documents are selected.
- ⑦ Context is built and passed to the LLM.
- ⑧ LLM generates the answer.
- ⑨ Response is returned to the user.

Example



Benefits

- ✓ Higher recall & better relevance
- ✓ Robust across different query types
- ✓ Handles semantic + keyword queries
- ✓ Reduces missing of important info
- ✓ Improves answer quality

Challenges

- ✗ More complex to build
- ✗ Higher latency & cost
- ✗ Need good merging & reranking
- ✗ Requires multiple infrastructure components
- ✗ Harder to tune & optimize

Use Cases

- Enterprise search
- Customer support systems
- Research assistants
- Legal & compliance search
- Technical documentation Q&A
- Multi-source knowledge bases

Components

- **Query Processing:** Rewrite, expand, or route query
- **Retrievers:** Dense, Sparse, Graph, SQL, etc.
- **Merger:** Combine and deduplicate results
- **Reranker:** Reorder by relevance (e.g., Cross-Encoder)
- **Context Builder:** Prepare context for LLM
- **LLM (Generator):** Generate final response

Hybrid RAG Variants

Dense + Sparse

- Combine semantic (dense) and keyword (sparse) search
- Most common hybrid approach

Multi-Stage Hybrid

- Use one retriever first then another for refinement
- Improves precision

Router-based Hybrid

- Route query to best retriever based on intent or type
- Dynamic & adaptive

Ensemble Hybrid

- Combine multiple retrievers and rankers using weighted scoring
- Highest robustness

Best Practices

- Use a mix of Dense + Sparse retrieval
- Deduplicate before reranking
- Rerank with a strong cross-encoder
- Keep context concise and relevant
- Continuously evaluate & optimize
- Monitor latency, cost & quality

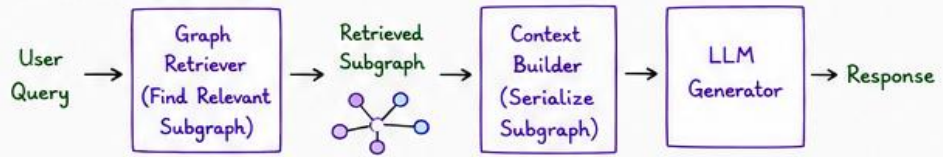
GraphRAG

(Graph-based Retrieval-Augmented Generation)

What is GraphRAG?

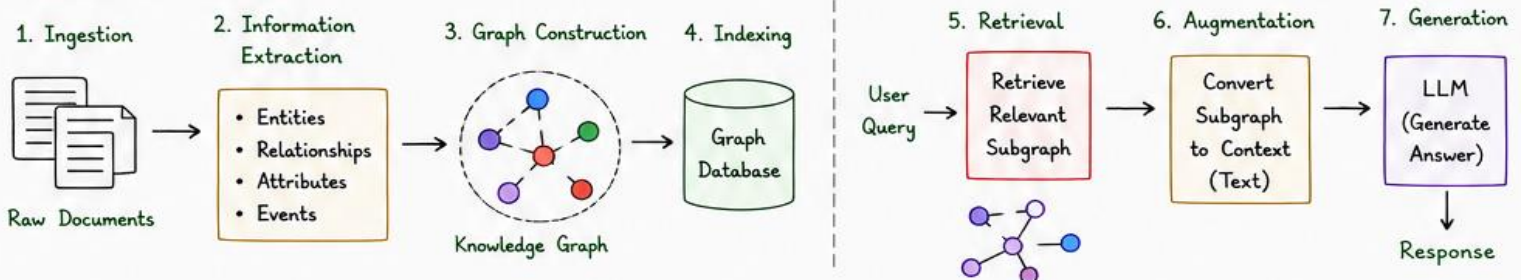
- Extends RAG by using knowledge graphs for retrieval and reasoning.
- Captures entities, relationships and context explicitly.
- Enables deeper understanding, connection discovery and better answers.
- Ideal for complex, multi-hop and relationship-heavy queries.

Core Components

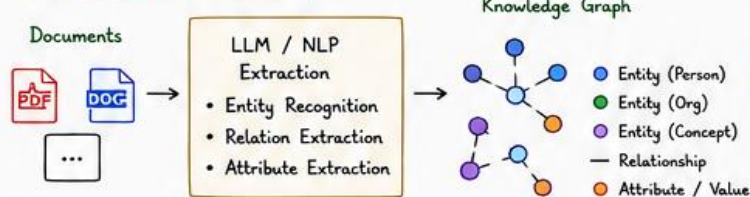


- Graph Retriever : Finds relevant nodes & edges from the graph.
- Context Builder : Converts subgraph into text for LLM.
- LLM Generator : Generates accurate answer with graph context.

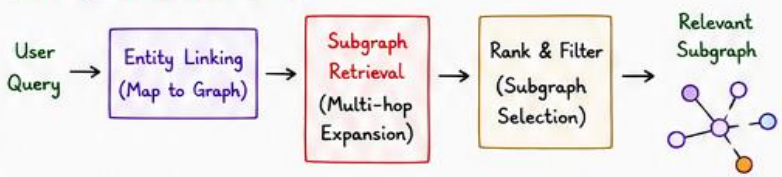
GraphRAG Pipeline



Graph Construction



Query in GraphRAG



Retrieval Strategies

- | 1. Local Search (Neighborhood) | 2. Path-based Search | 3. Community-based | 4. Semantic + Graph Hybrid | 5. Global Search |
|---|--|--|--|--|
| <ul style="list-style-type: none">• Start from entity• Traverse k-hop neighbors• Good for factual queries | <ul style="list-style-type: none">• Find relation paths between entities• Useful for reasoning & explanations | <ul style="list-style-type: none">• Retrieve based on communities or clusters• Captures broader context | <ul style="list-style-type: none">• Combine vector similarity with graph traversal• Improves recall and relevance | <ul style="list-style-type: none">• Explore entire graph (rarely)• For complex analytical queries |

Context Construction

- Serialize subgraph to LLM-friendly text.
- Include entities, relationships, attributes and summaries.
- Add natural language descriptions if available.
- Remove noise and keep relevant parts.

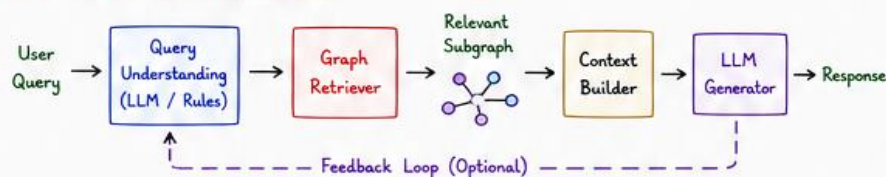
GraphRAG Variants

- | Naive GraphRAG | PathRAG | CommunityRAG | Hybrid GraphRAG | Hierarchical GraphRAG |
|---|---|--|---|--|
| <ul style="list-style-type: none">• Single-hop retrieval• Direct neighbors• Simple & fast | <ul style="list-style-type: none">• Path / multi-hop retrieval• Better reasoning | <ul style="list-style-type: none">• Community based subgraph retrieval• Broader context | <ul style="list-style-type: none">• Graph + Vector Hybrid• High accuracy | <ul style="list-style-type: none">• Coarse to fine retrieval• Scalable for large graphs |

Graph Databases

- | • Neo4j | • Memgraph |
|------------------|----------------------|
| • Amazon Neptune | • JanusGraph |
| • TigerGraph | • DGL / Graph Frames |
| • ArangoDB | |

Data Flow in GraphRAG



Advantages

- ✓ Comprehends explicit relationships
- ✓ Handles complex, multi-hop queries
- ✓ Better accuracy and explainability
- ✓ Reduces hallucinations
- ✓ Great for interconnected data
- ✓ Supports rich, contextual answers

Challenges

- ✗ Complex pipeline
- ✗ Graph construction cost
- ✗ Requires high quality extraction
- ✗ Scalability for very large graphs
- ✗ Harder to maintain

Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Relevance (Faithfulness)
- F1 / Exact Match
- Path Accuracy
- Hallucination Rate
- Latency & Throughput

Common Use Cases

- Enterprise Knowledge Bases
- Research & Academic Data
- Legal & Compliance
- Healthcare & Life Sciences
- Recommendation & Personalization
- Fraud Detection & Investigation

Best Practices

- ✓ Use high-quality extraction models
- ✓ Build clean and meaningful graphs
- ✓ Choose right retrieval strategy
- ✓ Re-rank and filter subgraphs
- ✓ Keep context concise but rich
- ✓ Continuously evaluate and improve

Future Directions

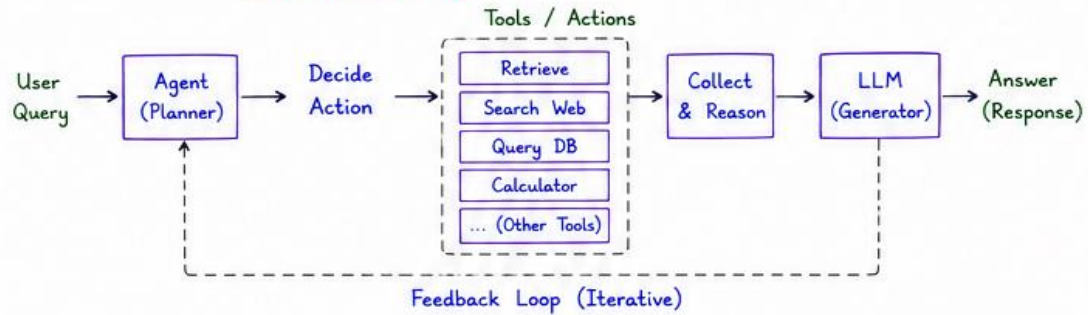
- Dynamic / Incremental GraphRAG
- LLMs that natively query graphs
- Better automatic graph construction
- Integration with agents
- Scalable, real-time GraphRAG

Agentic RAG

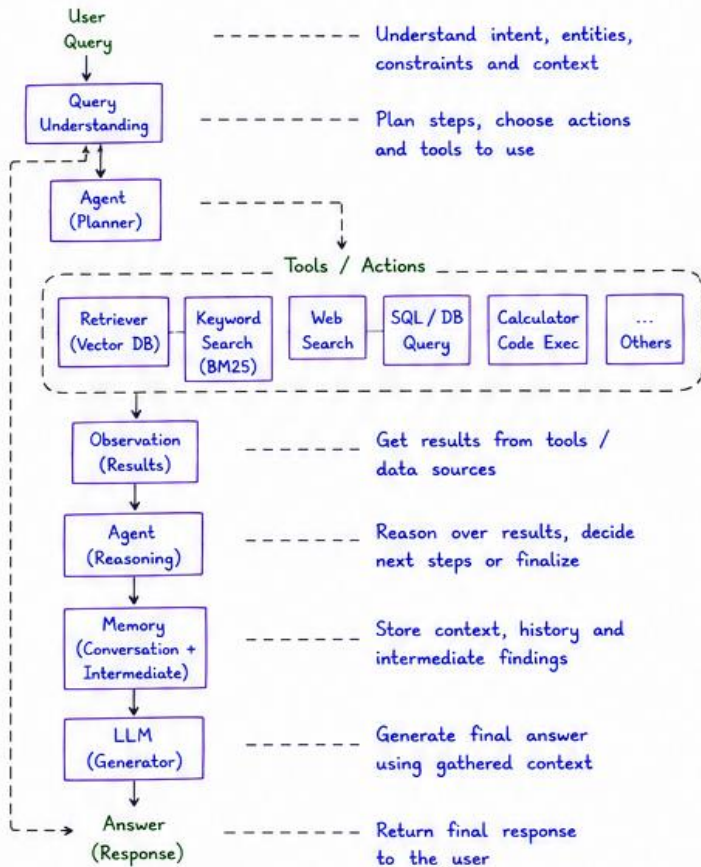
What is Agentic RAG?

- Uses an agent (LLM) to reason, plan and take actions.
- Dynamically decides what information to retrieve, from where and how.
- Iteratively retrieves, reasons and generates better answers.
- Best for complex, multi-hop and open-ended questions.

Overview (Flow)



Architecture (Flow Diagram)



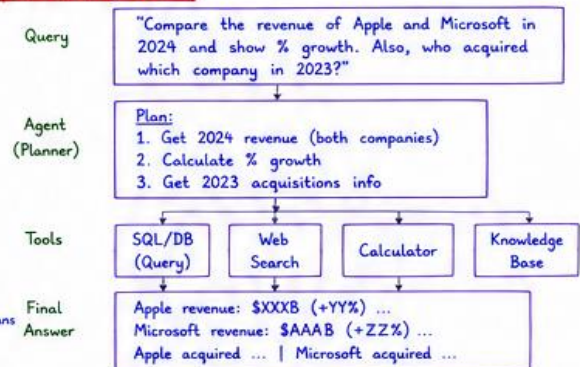
Key Capabilities

- Autonomous decision making (what to retrieve, from where, how many times)
- Multi-step reasoning and iterative retrieval
- Dynamic tool usage (search, DB, calculator, code, APIs, etc.)
- Maintains memory of conversation and intermediate results
- Adapts to complex, long-context, and evolving queries

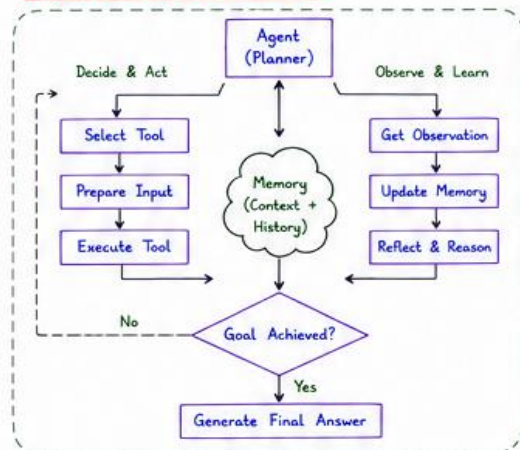
How it Works (Step-by-Step)

- ① User provides a query.
- ② Agent understands the query and plans the next action.
- ③ Agent selects and calls the appropriate tool(s).
- ④ Results (observations) are returned from the tool(s).
- ⑤ Agent reasons over the results and decides next action.
- ⑥ Steps ③-⑤ repeat until the agent has enough information.
- ⑦ Agent builds final context and passes it to the LLM.
- ⑧ LLM generates the final answer.
- ⑨ Response is returned to the user.

Example (Mini Flow)



Agent Loop (Detailed)



Components

- Agent (Planner): Decides actions, plans steps and controls the flow.
- Tools / Actions: External functions or data sources the agent can use.
- Retriever: Fetches relevant documents from knowledge base (vector DB).
- Memory: Stores conversation, context, intermediate steps and observations.
- LLM (Generator): Generates final answer using the accumulated context.

Benefits

- ✓ Handles complex, multi-hop queries
- ✓ More accurate and context-aware
- ✓ Flexible and extensible
- ✓ Can use multiple data sources
- ✓ Adapts and improves iteratively

Challenges

- ✗ More complex to build
- ✗ Higher latency & cost
- ✗ Harder to debug
- ✗ Risk of irrelevant loops
- ✗ Needs good tool design and guardrails

Use Cases

- Enterprise knowledge assistants
- Research and analytics
- Customer support (complex queries)
- Legal & compliance research
- Data analysis with DB + docs
- Technical troubleshooting
- Multi-source report generation
- Code / API documentation Q&A
- Market / competitor intelligence

Best Practices

- Design clear tools with good descriptions.
- Provide the agent with high-quality, relevant data.
- Use guardrails to prevent irrelevant loops.
- Limit the number of iterations or set stop conditions.
- Continuously evaluate and monitor agent behavior.

Key Features (At a Glance)

- ✓ Autonomous & goal-driven
- ✓ Iterative retrieval & reasoning
- ✓ Multi-tool orchestration
- ✓ Memory & context aware
- ✓ Produces high-quality, trustworthy answers

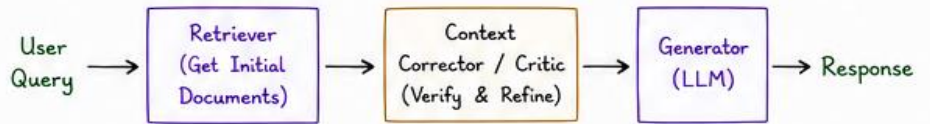
Corrective RAG (CRAG)

(Corrective Retrieval-Augmented Generation)

What is Corrective RAG (CRAG)?

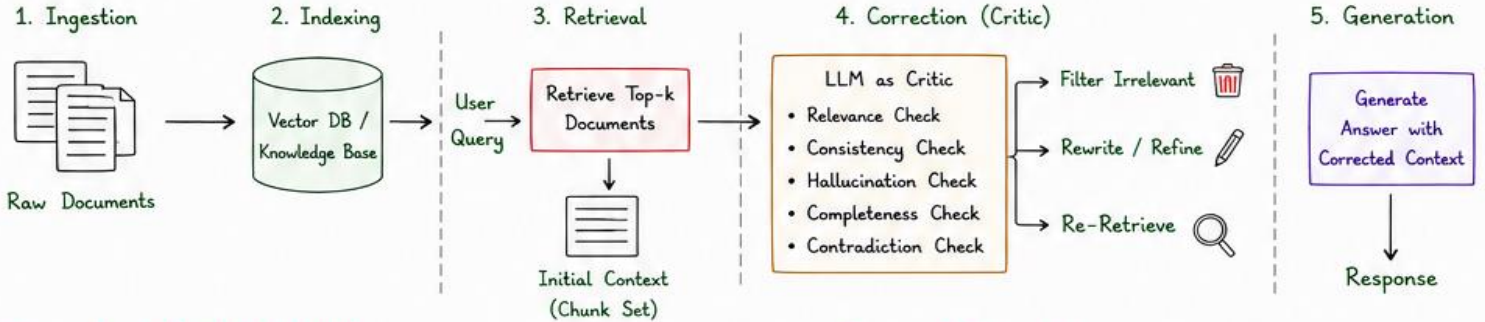
- Improves upon traditional RAG by verifying and correcting retrieved context before generation.
- Detects irrelevant, incomplete or inconsistent information.
- Uses LLM as a critic to refine context, re-retrieve or filter.
- Produces more accurate, reliable and trustworthy answers.

Core Components

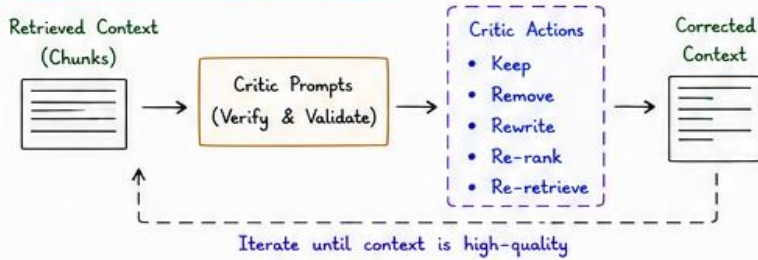


- Retriever** : Fetches relevant documents.
- Corrector/Critic** : Evaluates, filters, or rewrites context for correctness.
- Generator** : Generates final answer using corrected context.

CRAG Pipeline



Correction (Critic) Workflow



Types of Corrections

- | | | |
|--|---|---|
| 1. Relevance Correction <ul style="list-style-type: none">Remove irrelevant chunks. | 2. Consistency Correction <ul style="list-style-type: none">Fix contradictions or conflicting facts. | 3. Completeness Correction <ul style="list-style-type: none">Add missing but necessary info. |
| 4. Hallucination Prevention <ul style="list-style-type: none">Remove or replace unsupported claims. | 5. Noise Reduction <ul style="list-style-type: none">Remove redundant or noisy content. | 6. Re-Retrieval <ul style="list-style-type: none">If context is weak, retrieve again with better strategy. |

Retrieval Strategies in CRAG

- | | | | | |
|--|---|---|--|---|
| Initial Retrieval <ul style="list-style-type: none">Dense / Sparse retrievalHybrid retrieval | Multi-Query Retrieval <ul style="list-style-type: none">Generate multiple queries and retrieve broader context | Adaptive Retrieval <ul style="list-style-type: none">Critic decides if more context is needed and re-retrieves | Step-back Retrieval <ul style="list-style-type: none">Retrieve higher-level concepts for better context | Iterative Retrieval <ul style="list-style-type: none">Retrieve → Critic → Re-retrieve loop |
|--|---|---|--|---|

Context Correction Techniques

- LLM as Critic (Self-evaluation)
- Prompt-based Validation
- Cross-checking with external knowledge
- Contradiction detection
- Summarization & compression of context

Advantages

- Reduces hallucinations
- Improves factual accuracy
- Ensures consistency & completeness
- Adapts to complex queries
- Boosts reliability & trust

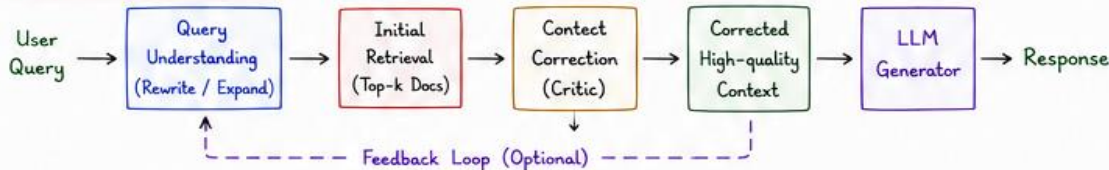
Limitations

- Higher latency (extra steps)
- More LLM calls → higher cost
- Complex to implement
- Depends on quality of critic prompts
- Not always perfect corrections

When to Use CRAG?

- High-stakes domains (Legal, Medical, Finance)
- When retrieval results are noisy or inconsistent
- Complex multi-hop or open-domain queries
- When accuracy and reliability are critical

Data Flow in CRAG



Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Correctness (Exact / F1)
- Faithfulness (Factual Consistency)
- Hallucination Rate
- Noise Reduction Rate
- Latency & Cost

CRAG Variants

- | | | | | |
|---|---|--|---|--|
| Self-Corrective RAG <ul style="list-style-type: none">Uses LLM to self-evaluate and correct its own context. | Iterative CRAG <ul style="list-style-type: none">Multiple correction cycles until context quality is acceptable. | Multi-Critic CRAG <ul style="list-style-type: none">Uses multiple critics (e.g., relevance, consistency, factuality). | Knowledge-Enhanced CRAG <ul style="list-style-type: none">Uses external knowledge graphs or rules for better correction. | Agentic CRAG <ul style="list-style-type: none">LLM agent decides when to retrieve, correct and re-generate. |
|---|---|--|---|--|

Best Practices

- Use high-quality retrievers
- Design strong critic prompts
- Set thresholds for filtering / re-retrieval
- Monitor correction loops
- Evaluate continuously
- Balance quality, latency & cost

Use Cases

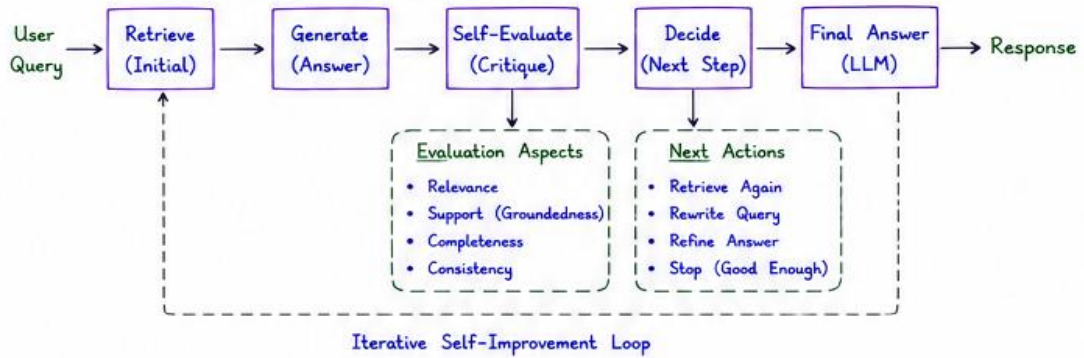
- Enterprise Search
- Question Answering Systems
- Chatbots / Virtual Assistants
- Research & Analysis
- Compliance & Risk Management
- Technical Documentation Search

Self-RAG

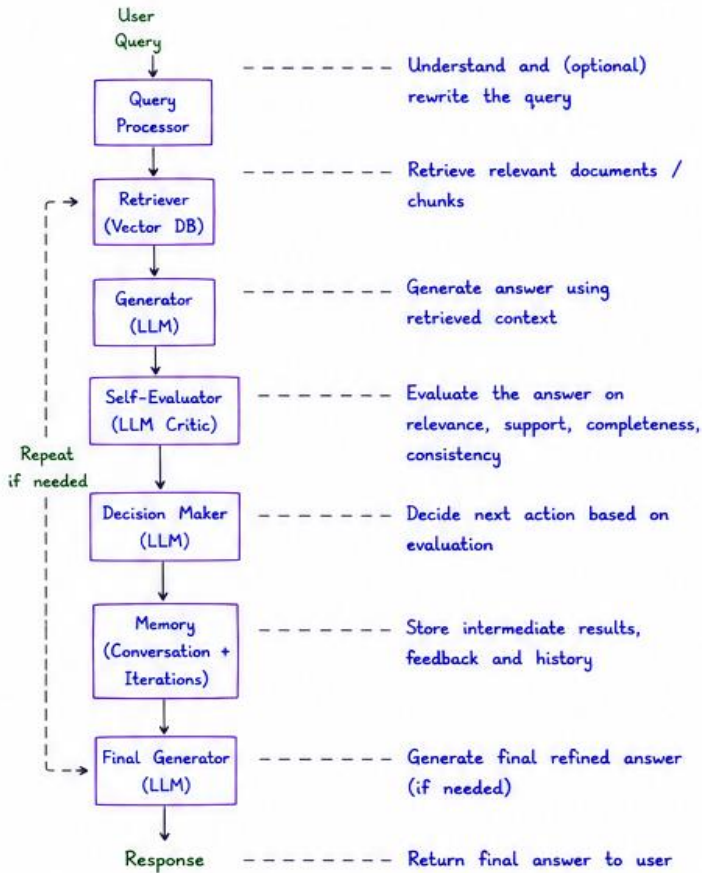
What is Self-RAG?

- The model (LLM) critiques its own retrieval and generation.
- Uses self-reflection to improve answers iteratively.
- Decides whether to retrieve again, rewrite, refine, or stop.
- Leads to higher accuracy and reliability.

Overview (Flow)



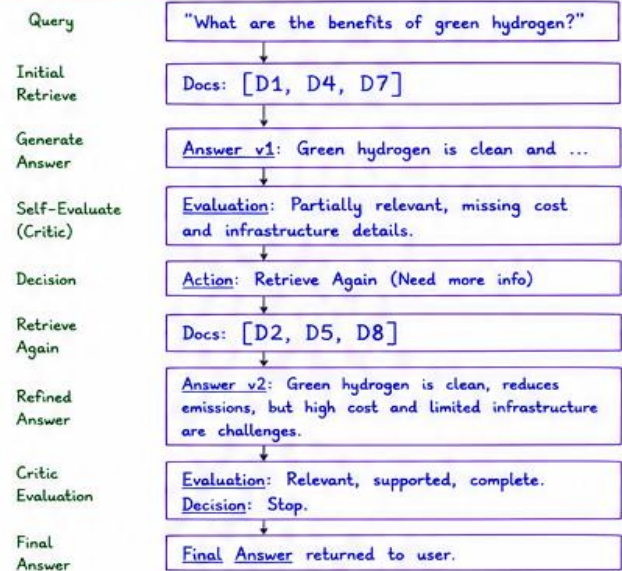
Architecture (Flow Diagram)



How it Works (Step-by-Step)

- ① User asks a question.
- ② System retrieves initial relevant documents.
- ③ LLM generates an answer using the retrieved context.
- ④ LLM (critic) evaluates the answer:
 - Is the answer relevant to the question?
 - Is it well-supported by the context?
 - Is anything missing or inconsistent?
- ⑤ Based on evaluation, LLM decides the next action:
 - If good enough → Stop.
 - Else → Retrieve again / Rewrite query / Refine answer.
- ⑥ Steps ②-⑤ repeat until a satisfactory answer is produced.
- ⑦ Final answer is returned to the user.

Example (Mini Flow)



Evaluation Aspects (Self-Evaluation)

- ✓ **Relevance:** Does the answer address the question?
- ✓ **Support (Groundedness):** Is the answer supported by the retrieved context?
- ✓ **Completeness:** Are all important points covered?
- ✓ **Consistency:** Is the answer logically consistent?
- ✓ **Usefulness:** Would this answer be helpful to the user?

Benefits

- ✓ Improves accuracy and reliability
- ✓ Reduces hallucinations
- ✓ Adapts to complex and ambiguous queries
- ✓ Self-corrects and refines iteratively
- ✓ Better user trust and satisfaction

Challenges

- ✗ More computational cost (multiple iterations)
- ✗ Higher latency
- ✗ Needs good evaluation prompts / criteria
- ✗ Risk of infinite loops (needs safeguards)
- ✗ Harder to debug and monitor

Use Cases

- Research assistants
- Knowledge-intensive QA
- Technical troubleshooting
- Legal & compliance search
- Healthcare & scientific domains
- Education & adaptive tutoring

Key Components

- **Retriever:** Fetches relevant documents
- **Generator (LLM):** Produces answers
- **Self-Evaluator (LLM Critic):** Evaluates the output
- **Decision Maker (LLM):** Decides next step
- **Memory:** Stores history and intermediate results
- **Tools (Optional):** Web search, calculators, APIs, etc.

Best Practices

- Use clear evaluation rubrics for the critic
- Set max iteration limits to avoid loops
- Combine with strong retrieval (hybrid if possible)
- Keep track of context and previous attempts
- Log evaluations and decisions for monitoring

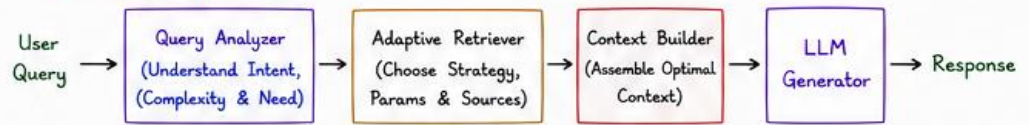
Adaptive RAG

(Adaptive Retrieval-Augmented Generation)

What is Adaptive RAG?

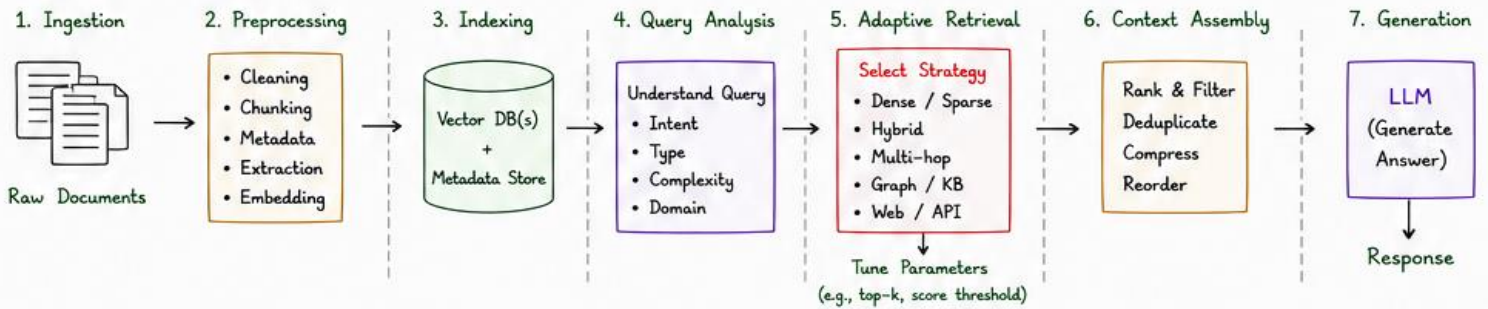
- Dynamically adapts retrieval strategy based on the query type and context need.
- Selects the best retrieval path, params and tools at runtime.
- Optimizes for accuracy, latency and cost.
- Continuously learns from feedback and improves over time.

Core Components

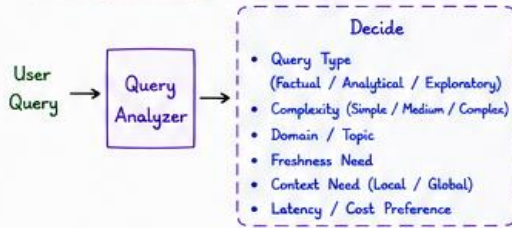


- Query Analyzer : Analyzes query to determine intent, complexity and info need.
- Adaptive Retriever : Chooses the best retrieval strategy, sources and parameters.
- Context Builder : Builds the most relevant and compact context.
- LLM Generator : Generates the final answer using the adaptive context.

Adaptive RAG Pipeline



Query Analysis (Decisions)



Adaptive Retrieval Strategies

Dense Retrieval	Sparse Retrieval	Hybrid Retrieval	Multi-hop Retrieval	Graph Retrieval	External Retrieval
• Vector similarity search • Best for semantic queries	• BM25 / TF-IDF • Best for keyword or exact match	• Combines dense + sparse • Better recall & precision	• Iteratively retrieve using intermediate results • Good for complex questions	• Traverse knowledge graphs • Captures relationships & connections	• Web / API / DB • Real-time or fresh information

Adaptive Decisions (Examples)

Query Type / Need	Strategy Chosen	Parameters Adapted
Simple factual	Dense	top-k = 5, low threshold
Keyword match	Sparse	BM25, k1 = 1.2
Complex reasoning	Multi-hop + Hybrid	top-k = 10, 2-hop
Relationship query	Graph Retrieval	Depth = 2, Path scoring
Fresh information	Web + Vector	Recency filter applied
Large context need	Hybrid + Rerank	top-k = 20, Rerank by LLM

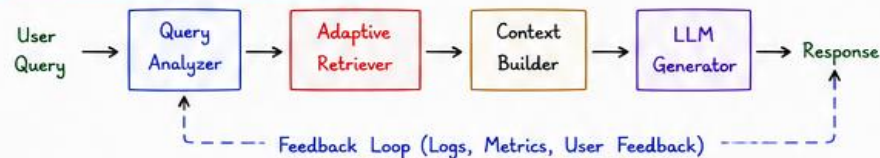
Context Building Techniques

- Rank by relevance score
- Deduplicate overlapping chunks
- Compress (LLM / Extractive summaries)
- Reorder (lost-in-the-middle mitigation)
- Assemble within token budget
- Add metadata & citations

Learning & Adaptation

- Log query, strategy, results & feedback
- Evaluate (relevance, answer quality)
- Learn which strategy works best
- Update routing rules & parameters
- Continuous improvement over time

Data Flow in Adaptive RAG



Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Relevance (Faithfulness)
- Answer Correctness (Exact / F1)
- Hallucination Rate
- Noise Reduction Rate
- Latency
- Throughput
- Cost
- User Satisfaction

Adaptive RAG Variants

Self-Adaptive RAG	Learned Adaptive RAG	Cost-Aware Adaptive RAG	Domain-Adaptive RAG	User-Adaptive RAG
• System decides strategy end-to-end • No manual routing	• ML model learns to select best strategy • Reinforcement / Supervised	• Balances accuracy, latency and cost • Optimizes resources	• Adapts based on domain / dataset characteristics	• Adapts to user preferences & history • Personalized retrieval

Advantages

- ✓ Best strategy is chosen automatically
- ✓ Higher accuracy and relevance
- ✓ Better handling of diverse queries
- ✓ Optimizes latency and cost
- ✓ Continuously improves with feedback

Challenges

- ✗ Complex to design and tune
- ✗ Requires good monitoring & logging
- ✗ Needs quality feedback signals
- ✗ Higher initial engineering effort
- ✗ Risk of over-optimization

Best Practices

- Start simple, then add adaptive layers
- Monitor strategies performance
- Keep context concise but sufficient
- Use reranking & reordering
- Continuously evaluate & iterate

Use Cases

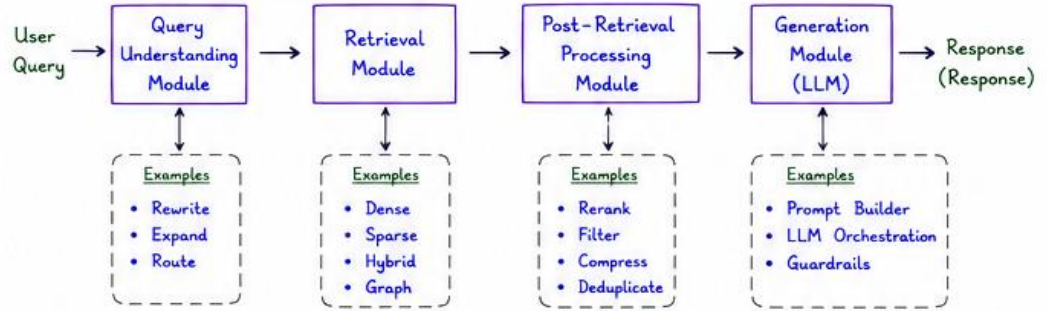
- Enterprise Knowledge Assistants
- Research & Academic Search
- Legal & Compliance Systems
- Customer Support Bots
- Complex Analytical Applications

Modular RAG

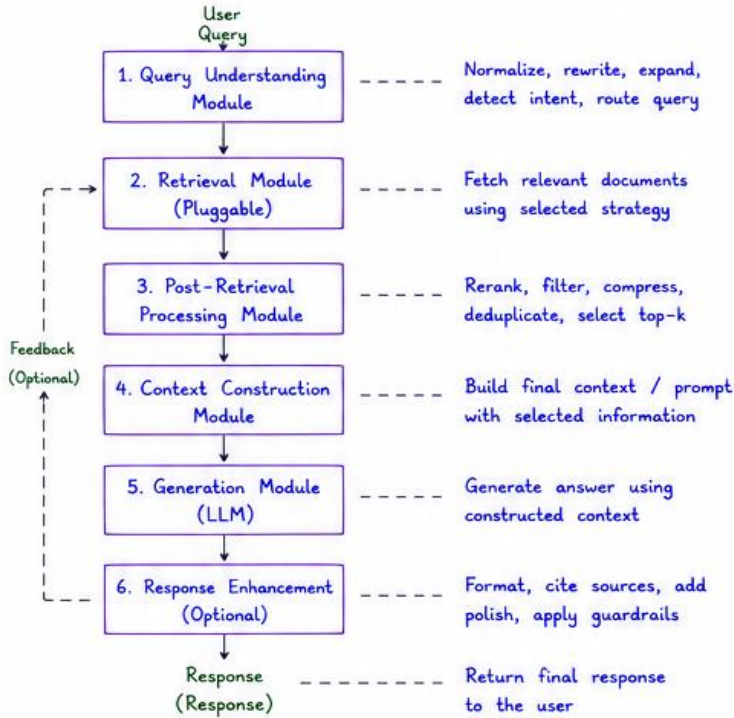
What is Modular RAG?

- Breaks the RAG pipeline into independent modules with well-defined interfaces.
- Each module can be replaced, upgraded or customized easily.
- Promotes flexibility, reusability, and maintainability.
- Helps adapt to different use cases and data sources.

Overview (Flow)



Architecture (Flow Diagram)



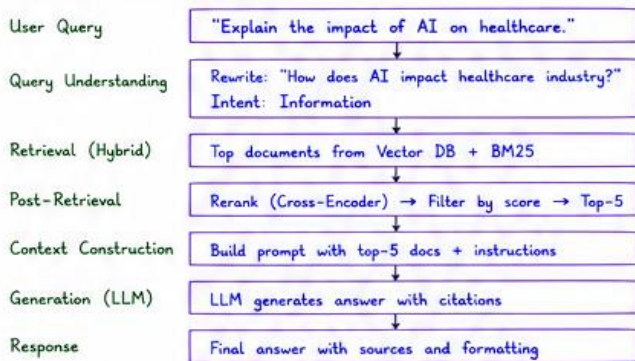
Key Benefits

- ✓ Highly flexible and customizable
- ✓ Swap or upgrade any module independently
- ✓ Supports multiple retrieval and LLM backends
- ✓ Better maintainability and scalability
- ✓ Easier testing and debugging
- ✓ Adapts well to different domains and use cases

How it Works (Step-by-Step)

- ① User submits a query.
- ② Query Understanding module interprets and prepares the query.
- ③ Retrieval module fetches relevant documents.
- ④ Post-Retrieval module refines and selects the best information.
- ⑤ Context Construction module builds the final prompt.
- ⑥ Generation module (LLM) produces the answer.
- ⑦ Response Enhancement (optional) improves format & adds sources.
- ⑧ Final response is returned to the user.

Example (Mini Flow)



Modules and Their Responsibilities

Module	Responsibility
1. Query Understanding	Understand intent, rewrite, expand, route, detect entities
2. Retrieval (Pluggable)	Fetch relevant docs (Dense / Sparse / Hybrid / Graph)
3. Post-Retrieval Processing	Rerank, filter, deduplicate, compress, select top-k
4. Context Construction	Assemble final context / prompt for the LLM
5. Generation (LLM)	Generate answer using the context
6. Response Enhancement	Format response, add citations, apply guardrails / safety

Common Retrieval Strategies (Module 2)

Dense Retrieval (Vector DB)	Sparse Retrieval (BM25)	Hybrid Retrieval	Graph Retrieval
• Semantic search • FAISS, Pinecone, Milvus, Weaviate	• Keyword based • Elasticsearch, OpenSearch	• Combine dense + sparse results • Better recall and precision	• Use knowledge graph structure • Good for relational queries

Post-Retrieval Processing Options (Module 3)

Reranking	Filtering	Compression	Deduplication
• Reorder using cross-encoder • Improves relevance	• By metadata • By score • By domain / date	• Summarize • Extract key points • Reduce tokens	• Remove duplicate or near-duplicate chunks

Use Cases

- Enterprise knowledge assistants
- Customer support bots
- Research and analytics
- Legal & compliance search
- Education & adaptive learning
- Healthcare information systems

Challenges

- ✗ Designing module interfaces
- ✗ Choosing the right modules
- ✗ Higher engineering effort initially
- ✗ Module interaction complexity
- ✗ Latency across multiple modules
- ✗ Cost management (multiple services)

Best Practices

- Keep modules loosely coupled
- Define clear input / output schemas
- Log and monitor each module
- Test modules independently
- Use caching where possible
- Iteratively improve each module

Key Components (Tech Examples)

- Vector DBs: FAISS, Pinecone, Milvus, Weaviate
- Search: Elasticsearch, OpenSearch, BM25
- Rerankers: Cohere Rerank, Cross-Encoder
- LLMs: OpenAI, Llama, Mistral, Claude
- Orchestration: LangChain, LlamaIndex
- Tools: Web Search, Calculator, APIs, DBs

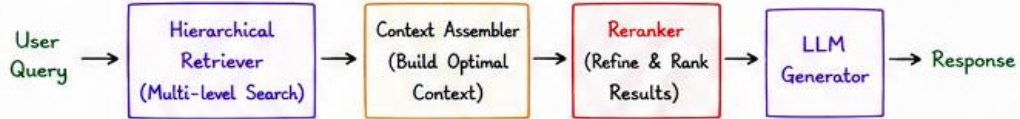
Hierarchical RAG

(Hierarchical Retrieval-Augmented Generation)

What is Hierarchical RAG?

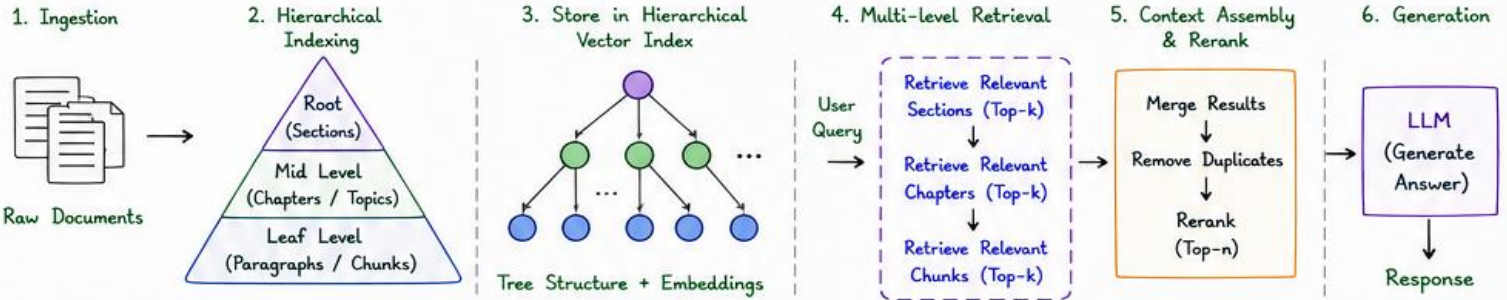
- Organizes knowledge in a hierarchy (e.g., sections → chapters → paragraphs).
- Retrieves at multiple levels for better context and efficiency.
- Improves relevance, reduces noise and lowers cost.
- Ideal for large, structured documents and enterprise data.

Core Components

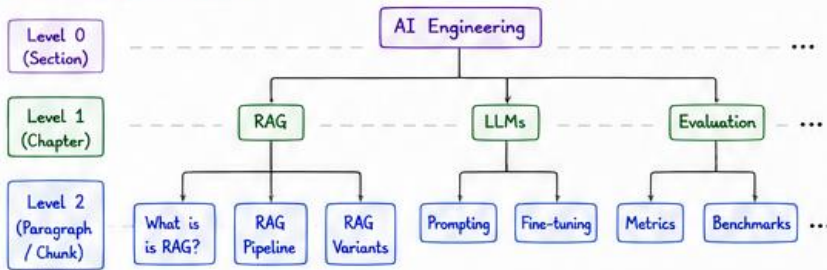


- Hierarchical Retriever** : Searches across levels (e.g., section → chunk).
- Context Assembler** : Combines the most relevant pieces from different levels.
- Reranker** : Reorders and filters for final context.
- LLM Generator** : Generates answer using the hierarchical context.

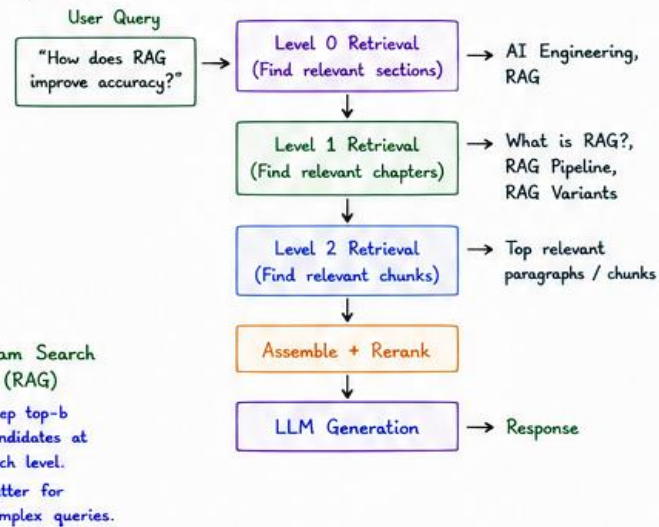
Hierarchical RAG Pipeline



Hierarchy Example



Retrieval Process (Example)



Retrieval Strategies

- | Top-down Retrieval | Bottom-up Retrieval | Hybrid Retrieval | Selective Expansion | Beam Search (RAG) |
|---|---|--|---|--|
| <ul style="list-style-type: none">Start from top level and go down.Good for broad queries. | <ul style="list-style-type: none">Search leaf nodes first, then aggregate upwards.Good for specific queries. | <ul style="list-style-type: none">Combine top-down and bottom-up.Balances recall and precision. | <ul style="list-style-type: none">Expand only relevant branches.Improves efficiency and reduces noise. | <ul style="list-style-type: none">Keep top-b candidates at each level.Better for complex queries. |

Indexing & Data Organization

- Split documents hierarchically (Section → Chapter → Paragraph → Chunk).
- Create embeddings at each level.
- Store in a hierarchical vector database (e.g., Tree-based index).
- Maintain parent-child relationships.

Context Assembly Techniques

- Concatenate from multiple levels.
- Add summaries at higher levels.
- Use headings / breadcrumbs.
- Remove redundancy.
- Keep within token budget.

Use Cases

- Enterprise Knowledge Bases
- Technical Documentation Search
- Legal & Compliance Search
- Research Papers / Academic Search
- Customer Support Knowledge
- E-commerce Product Knowledge
- Large Manuals & SOPs

Advantages

- ✓ Better relevance and precision
- ✓ Handles long documents efficiently
- ✓ Reduces noise and hallucinations
- ✓ Scalable for enterprise knowledge
- ✓ Cost-effective (fewer tokens)
- ✓ Supports complex & multi-hop queries

Limitations

- ✗ Complex to design and implement
- ✗ Requires good hierarchy structure
- ✗ More indexing overhead
- ✗ Latency may increase with levels
- ✗ Needs tuning of retrieval depth

Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Relevance (Faithfulness)
- Answer Correctness (Exact / F1)
- Hallucination Rate
- Latency & Throughput
- Cost (Tokens / \$)

Best Practices

- ✓ Design meaningful hierarchy
- ✓ Use summaries at higher levels
- ✓ Tune top-k at each level
- ✓ Apply reranking at the end
- ✓ Monitor and evaluate continuously
- ✓ Update hierarchy as content grows

Hierarchical RAG Variants

- | Summary-based Hierarchical RAG | Graph + Hierarchical RAG | Adaptive Depth RAG | Caching Hierarchical RAG |
|--|---|--|--|
| <ul style="list-style-type: none">Use summaries at each level.Faster and cheaper. | <ul style="list-style-type: none">Combine hierarchy with knowledge graph.Better for relations. | <ul style="list-style-type: none">Dynamically decide how deep to retrieve.Save cost & time. | <ul style="list-style-type: none">Cache embeddings & results per level.Improve speed. |

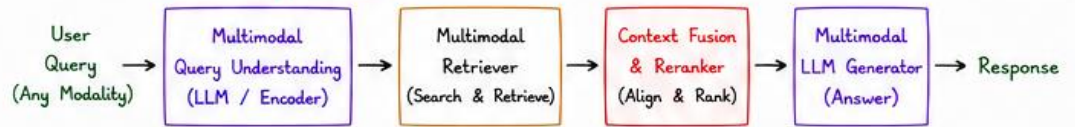
Multimodal RAG

(Multimodal Retrieval-Augmented Generation)

What is Multimodal RAG?

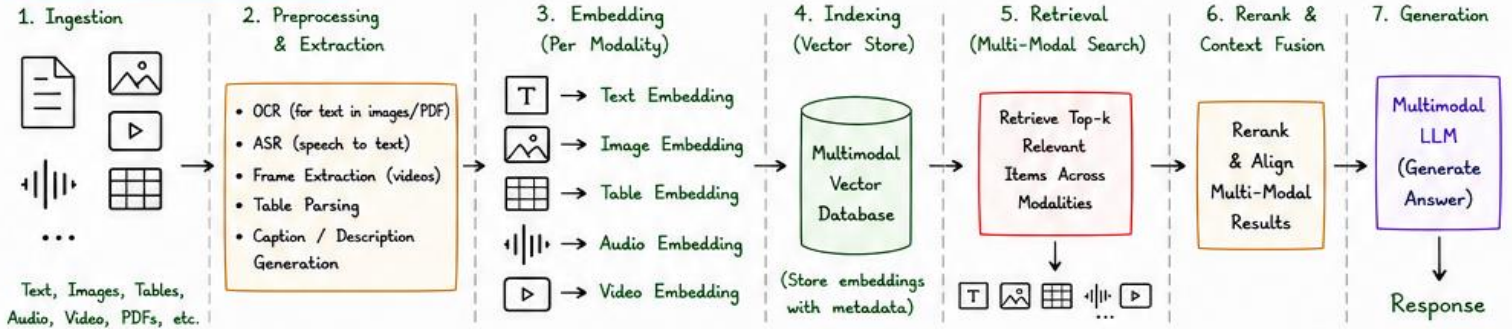
- Extends traditional RAG to process multiple data modalities (text, images, tables, audio, videos, etc.).
- Retrieves and reasons over information from different modalities.
- Provides richer context and more accurate answers.
- Useful for real-world applications where information exists in diverse formats.

Core Components



- Multimodal Query Understanding : Understands query across modalities.
- Multimodal Retriever : Retrieves relevant info from multiple modalities.
- Context Fusion & Reranker : Aligns, fuses and ranks retrieved content.
- Multimodal LLM Generator : Generates final answer using multimodal context.

Multimodal RAG Pipeline



Supported Modalities & Examples

- T Text (Articles, Docs, Notes)
- Image Images (Photos, Charts, Diagrams)
- Table Tables (Spreadsheets, Reports)
- Audio Audio (Podcasts, Meetings)
- Video Video (Lectures, Demonstrations)
- ...

Retrieval Strategies

- Early Fusion**
 - Combine embeddings of all modalities before retrieval.
 - Good for simple queries.
- Late Fusion**
 - Retrieve per modality and merge results.
 - Flexible and widely used.
- Hybrid Fusion**
 - Combines early + late fusion.
 - Balances accuracy and efficiency.
- Cross-Modal Retrieval**
 - Use one modality query to retrieve across other modalities.
 - Example: Image query → Text docs.

Query Types (Examples)

- Text Query → Find related images, tables
- Image Query → Find similar images, descriptions
- Audio Query → Find meeting notes, transcripts
- Video Query → Find key moments, summaries
- Mixed Query → "Explain this chart and related data" (image + text)

Context Fusion Techniques

- Concatenate text from multiple modalities.
- Use modality-specific encoders + alignment.
- Create unified representation for LLM.
- Preserve modality information with tags (e.g., [Image], [Table], [Audio]).

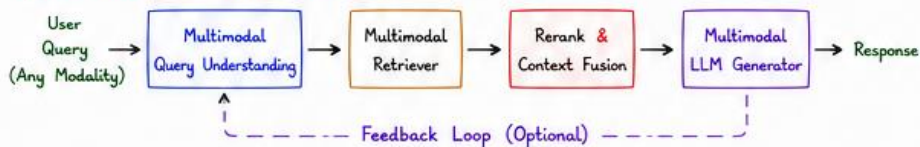
Multimodal LLMs (Examples)

- GPT-4o / GPT-4V
- Gemini 1.5 Pro
- Claude 3.5 (Vision)
- LLaVA
- MiniGPT-4
- BLIP-2
- InstructBLIP
- Florence-2
- Qwen-VL
- InternVL

Applications

- Research & Academic Search
- Medical Imaging + Reports
- Finance (Charts + Texts + Tables)
- Legal Document Analysis
- E-learning (Videos + Notes + Slides)
- Customer Support (Screenshots + Logs + Conversations)

Data Flow in Multimodal RAG



Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Relevance (Faithfulness)
- Answer Correctness (Exact / F1)
- Multimodal Alignment Score
- Hallucination Rate
- Latency
- Throughput
- Token Efficiency
- User Satisfaction

Advantages

- ✓ Handles real-world multimodal data
- ✓ Provides richer context
- ✓ Improves accuracy and completeness
- ✓ Cross-modal understanding
- ✓ Better user experience

Challenges

- ✗ Complex preprocessing & pipelines
- ✗ High computational cost
- ✗ Large storage for embeddings
- ✗ Modality alignment is difficult
- ✗ Requires powerful multimodal models

Best Practices

- Use high-quality embeddings per modality
- Choose right retrieval strategy for use case
- Rerank and filter before generation
- Keep context concise and structured
- Continuously evaluate and improve

Use Cases

- Enterprise Knowledge Search
- Multimodal Document Understanding
- Video Summarization + QA
- Image + Text QA Systems
- Multimodal Chatbots

Multimodal RAG Variants

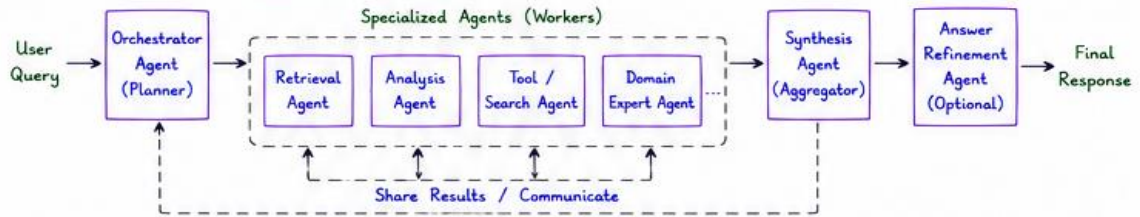
- | Text + Image RAG | Text + Table RAG | Audio + Text RAG | Video + Text RAG | All-Modality RAG | Streaming RAG |
|--|---|--|---|--|---|
| <ul style="list-style-type: none">• Retrieve images and texts• Good for visual explanations | <ul style="list-style-type: none">• Retrieve tables + text• Good for data analysis & reporting | <ul style="list-style-type: none">• Speech to text• Retrieve transcripts, summaries | <ul style="list-style-type: none">• Retrieve key frames, transcripts• Good for education, surveillance | <ul style="list-style-type: none">• Combine text, images, tables, audio, video• Maximum context | <ul style="list-style-type: none">• Real-time retrieval from live streams• Good for IoT, live events |

Multi-Agent RAG

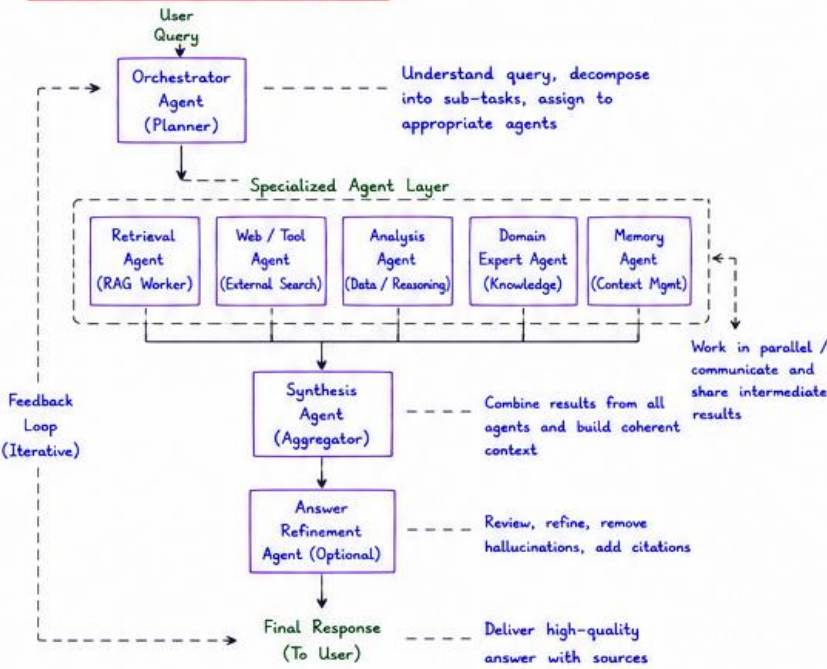
What is Multi-Agent RAG?

- Uses multiple specialized agents that collaborate to solve complex queries.
- Each agent has a specific role and expertise.
- Agents communicate and share information to produce better answers.
- Ideal for multi-hop, cross-domain and open-ended questions.

Overview (Flow)



Architecture (Flow Diagram)



How it Works (Step-by-Step)

- ① User submits a complex query.
- ② Orchestrator Agent understands the query and creates a plan.
- ③ Orchestrator assigns subtasks to relevant specialized agents.
- ④ Agents retrieve information, search web/tools, analyze data, and apply domain knowledge.
- ⑤ Agents communicate and share intermediate results.
- ⑥ Synthesis Agent aggregates the results and builds the final context.
- ⑦ Answer Refinement Agent reviews and improves the answer (checks quality, adds citations, fixes inconsistencies).
- ⑧ If needed, orchestrator iterates with agents for more info.
- ⑨ Final high-quality response is returned to the user.

Example (Mini Flow)

User Query	"Compare the impact of remote work on productivity and mental health across industries."
Orchestrator (Planner)	Plan: Identify subtopics → Assign to agents
Retrieval Agent	Fetches papers, reports, stats from Vector DB
Web / Tool Agent	Searches latest articles, industry reports, datasets
Analysis Agent	Analyzes data, extracts key insights, compares trends
Domain Expert Agent	Applies HR / Psychology domain knowledge
Synthesis Agent	Combines all results into a structured answer
Answer Refinement Agent	Checks accuracy, removes hallucinations, adds citations
Final Response	Comprehensive answer with citations and insights.

Types of Agents and Roles

Agent	Role / Responsibility
1. Orchestrator (Planner)	Understands query, breaks into subtasks, assigns tasks, controls workflow, manages iterations.
2. Retrieval Agent (RAG Worker)	Retrieves relevant documents from vector DB / knowledge base.
3. Web / Tool Agent	Searches web, calls APIs, uses tools and external sources.
4. Analysis Agent	Analyzes data, performs reasoning, summarization, calculations.
5. Domain Expert Agent	Provides domain-specific knowledge and expert reasoning.
6. Memory / Context Agent	Tracks conversation history and shared context across agents.
7. Synthesis / Aggregator Agent	Merges all results and builds coherent final context.
8. Answer Refinement Agent	Reviews, refines, validates and formats the final answer.

Benefits

- ✓ Handles complex, multi-hop queries
- ✓ Higher accuracy and reliability
- ✓ Leverages specialization of agents
- ✓ Scalable and extensible
- ✓ Reduces hallucinations
- ✓ Better for cross-domain tasks
- ✓ Supports parallel processing

Challenges

- ✗ More complex orchestration
- ✗ Higher latency and cost
- ✗ Communication overhead
- ✗ Needs good agent design
- ✗ Harder to debug and monitor
- ✗ Risk of inconsistent outputs

Use Cases

- Enterprise knowledge assistants
- Research & academic discovery
- Legal & compliance analysis
- Healthcare decision support
- Market / competitor intelligence
- Customer support (complex queries)
- Multi-document summarization

Key Components (Tech Examples)

- LLMs: GPT-4o, Claude, Llama 3, Mistral
- Vector DBs: Pinecone, Weaviate, Milvus, Qdrant
- Orchestration: LangChain, AutoGen, crewAI, Semantic Kernel
- Tools: SerpAPI, Tavily, Wolfram Alpha, APIs
- Frameworks: LangGraph, Haystack, LlamaIndex Agents
- Memory: Redis, MongoDB, Postgres

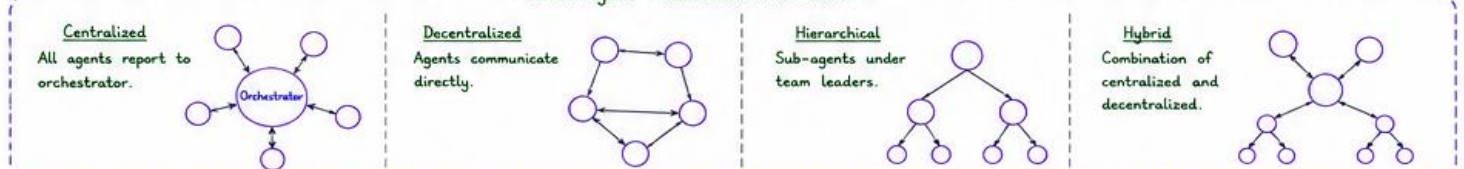
Best Practices

- Define clear roles and responsibilities
- Design good communication protocols
- Use structured outputs between agents
- Implement feedback and iteration
- Monitor and log agent interactions
- Evaluate each agent and the system continuously

Tips for Success

- Start with simple workflows
- Add agents gradually
- Use caching and indexing to reduce cost
- Ensure context is shared efficiently
- Keep humans in the loop for critical tasks

Multi-Agent Communication Patterns



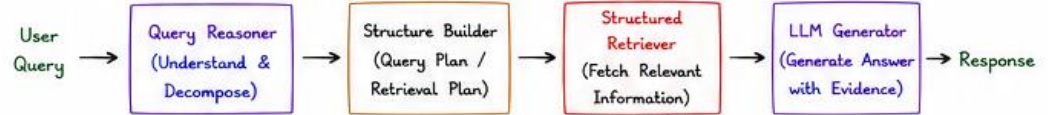
Reasoning-First / Structured Retrieval RAG

(Reasoning-First / Structured Retrieval-Augmented Generation)

What is Reasoning-First RAG?

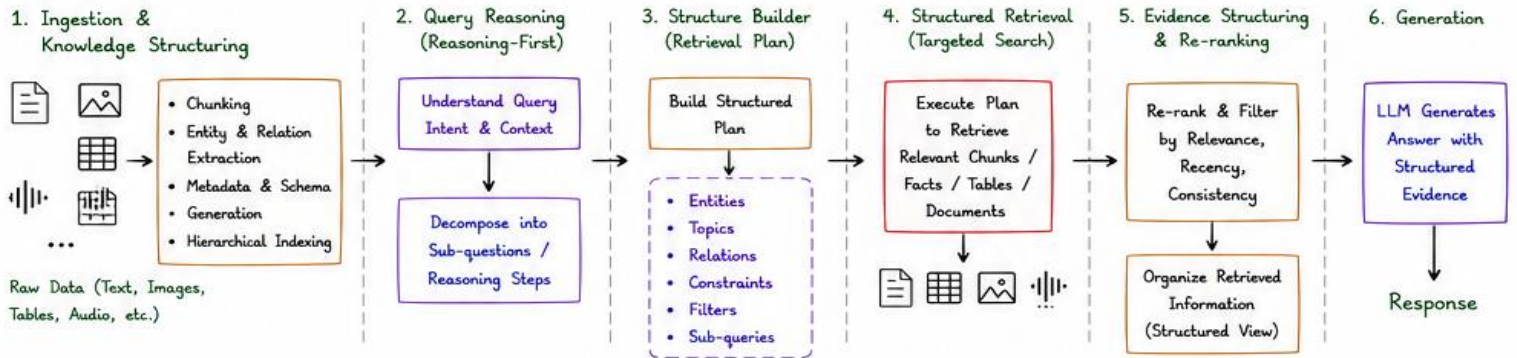
- Puts reasoning & planning before retrieval.
- LLM first understands, decomposes and reasons about the query.
- Then performs structured retrieval based on the reasoning.
- Leads to more accurate, relevant and comprehensive answers.
- Ideal for complex, multi-hop and knowledge-intensive queries.

Core Components



- Query Reasoner : Analyzes query intent, breaks into sub-questions / reasoning steps.
- Structure Builder : Builds a structured plan (topics, entities, relations, filters).
- Structured Retriever : Executes targeted retrieval using the structured plan.
- LLM Generator : Synthesizes final answer using retrieved structured information.

Reasoning-First RAG Pipeline



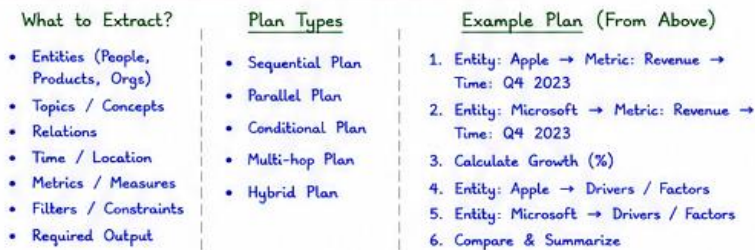
Query Reasoning (Examples)



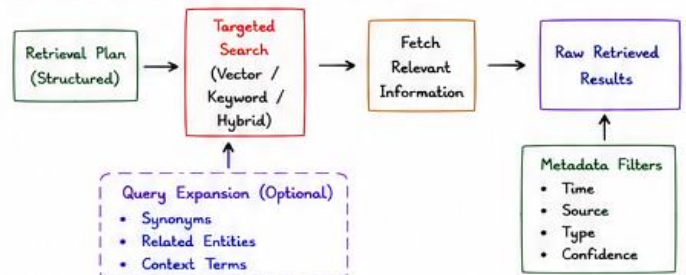
Retrieval Strategies

Entity-Centric	Relation-Centric	Schema-Aware	Logic-Driven
• Retrieve info about key entities • Good for factual queries	• Retrieve based on relationships between entities • Good for multi-hop queries	• Use table schema, metadata & structure • Good for tables & databases	• Retrieve based on logical conditions from plan • Good for analytical queries

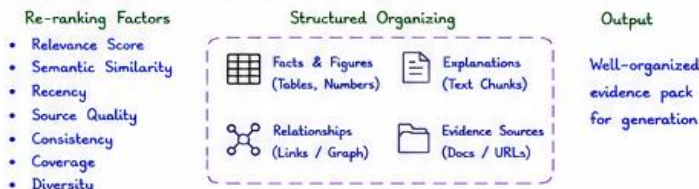
Structure Builder (Retrieval Plan Elements)



Structured Retrieval (How it Works)



Evidence Structuring & Re-ranking



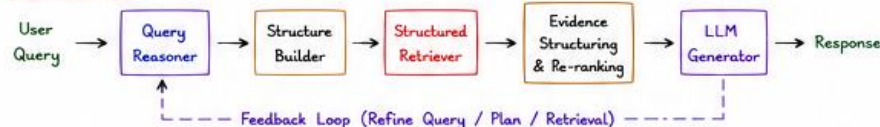
Benefits

- ✓ More accurate answers with better coverage
- ✓ Reduces hallucinations
- ✓ Better for complex & multi-hop reasoning
- ✓ Efficient retrieval by targeting what matters
- ✓ Improves interpretability & trust
- ✓ Scalable to enterprise knowledge bases

Use Cases

- Research & Academic Search
- Legal & Compliance Analysis
- Financial & Market Intelligence
- Technical Documentation Q&A
- Healthcare Information Retrieval
- Business Analytics & Reports

Data Flow



Evaluation Metrics

- Context Relevance (Recall@k)
- Answer Relevance (Faithfulness)
- Answer Correctness (Exact / F1)
- Reasoning Accuracy
- Coverage
- Hallucination Rate
- Latency
- User Satisfaction

Advantages

- ✓ Improves reasoning & answer quality
- ✓ More relevant & targeted retrieval
- ✓ Handles complex queries effectively
- ✓ Better use of context & structure
- ✓ Reduces noise and token usage
- ✓ Scalable with structured knowledge

Limitations

- ✗ More complex to design & implement
- ✗ Requires good reasoning capability
- ✗ Higher latency (extra reasoning step)
- ✗ Depends on quality of structure/schema
- ✗ Needs metadata & preprocessing

Best Practices

- Use strong reasoning model for decomposition
- Build clear retrieval plans
- Use hybrid search with filtering
- Rerank and filter aggressively
- Continuously evaluate and iterate
- Monitor and refine reasoning steps

Future Directions

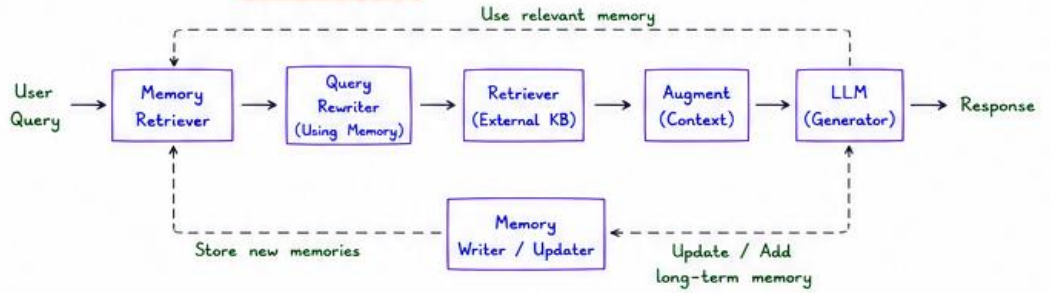
- Integrate with Agents & Tool Use
- Graph + Reasoning Fusion
- Learning-based Retrieval Planning
- Self-improving Reasoning Loops
- Dynamic Plan Adaptation in Real-time

Memory in RAG

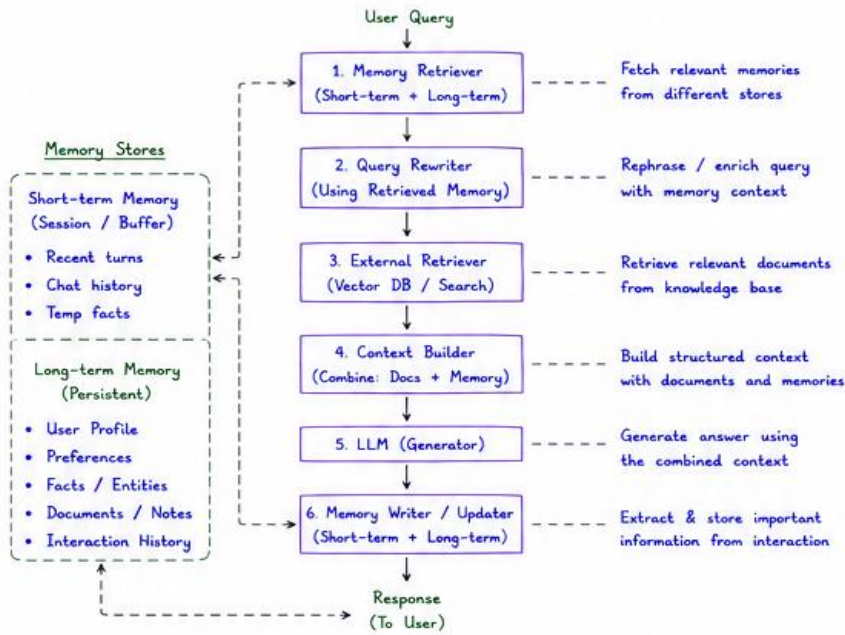
What is Memory in RAG?

- Adds memory to RAG so the system remembers past interactions, user preferences, and context.
- Enables more relevant, personalized and consistent answers over time.
- Memory can be short-term (session) or long-term (persistent).
- Helps in multi-turn conversations and follow-up queries.

Overview (Flow)



Architecture (Flow Diagram)



Types of Memory

Short-term Memory

- Session based (per conversation)
- Stores recent messages / turns
- Limited size (sliding window)
- Resets after session ends

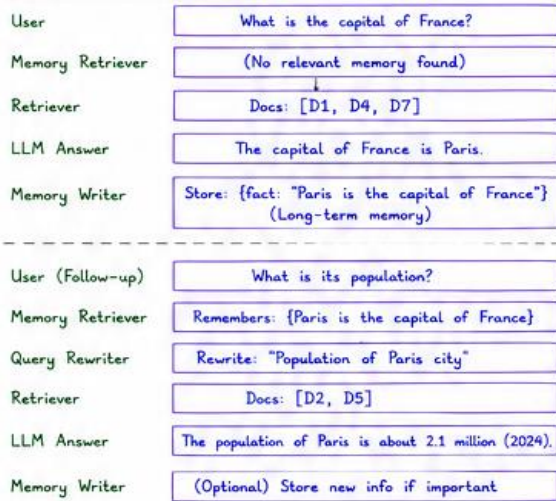
Long-term Memory

- Persistent across sessions
- Stores user facts, preferences, documents, notes
- Improves personalization
- Stored in DB / Vector DB

How it Works (Step-by-Step)

- ① User asks a question.
- ② Memory Retriever fetches relevant short-term and long-term memories.
- ③ Query Rewriter uses memory to understand context and rewrite query.
- ④ External Retriever fetches relevant documents / knowledge.
- ⑤ Context Builder combines memory + retrieved docs into a prompt.
- ⑥ LLM generates the answer.
- ⑦ Memory Writer extracts important info (facts, preferences, entities) and stores it in memory.
- ⑧ Response is returned to the user.

Example (Mini Flow)



Memory Stores and Technologies

Memory Type	Purpose	Examples / Technologies
Conversation Memory (Short-term)	Stores recent turns for context	Buffer, Sliding Window, Redis, LangChain ConversationBufferMemory
Entity Memory (Long-term)	Stores facts about entities / users	Vector DB (Pinecone, Weaviate), SQL DB, Graph DB (Neo4j)
Preference Memory (Long-term)	Stores user preferences	SQL DB, Key-Value Store, Vector DB
Document / Note Memory (Long-term)	Stores user uploaded notes / docs	Vector DB, Blob Storage + Metadata DB
Episodic Memory (Long-term)	Stores past interactions and events	SQL / NoSQL DB, Vector DB

Benefits

- ✓ Better contextual understanding
- ✓ Handles follow-up questions
- ✓ Personalized and adaptive responses
- ✓ Consistent experience across sessions
- ✓ Remembers user preferences and facts
- ✓ Improves answer accuracy

Challenges

- ✗ Memory management complexity
- ✗ Privacy and security concerns
- ✗ Storage cost for long-term memory
- ✗ Risk of outdated or irrelevant memory
- ✗ Need for summarization / pruning

Use Cases

- Customer support chatbots
- Personal AI assistants
- Research assistants
- Healthcare assistants
- E-learning & tutoring
- Legal & compliance search
- Knowledge management systems

Key Components

- **Memory Retriever:** Fetches relevant memories
- **Query Rewriter:** Uses memory to improve query
- **External Retriever:** Fetches relevant documents
- **Context Builder:** Merges memory + docs
- **LLM (Generator):** Generates final answer
- **Memory Writer:** Extracts & stores new memories

Best Practices

- Separate short-term and long-term memory
- Store only important and relevant info
- Use summaries to save space
- Regularly clean / prune outdated memory
- Respect privacy and user consent
- Monitor memory quality and relevance

Key Technologies

- LLMs: GPT-4o, Claude, Llama 3, Mistral
- Vector DBs: Pinecone, Weaviate, Milvus
- Databases: PostgreSQL, MongoDB, Redis
- Frameworks: LangChain, LlamaIndex
- Tools: Embeddings, Rerankers, Summarizers

Memory Types at a Glance



Working Memory

Holds current session context and recent turns. (Time-limited)



Semantic Memory

Stores facts, concepts, and general knowledge (long-term).



Episodic Memory

Stores past interactions, events and user history (long-term).



Procedural Memory

Stores user preferences, habits and settings (long-term).



Document Memory

Stores user documents, notes and uploaded files (long-term).

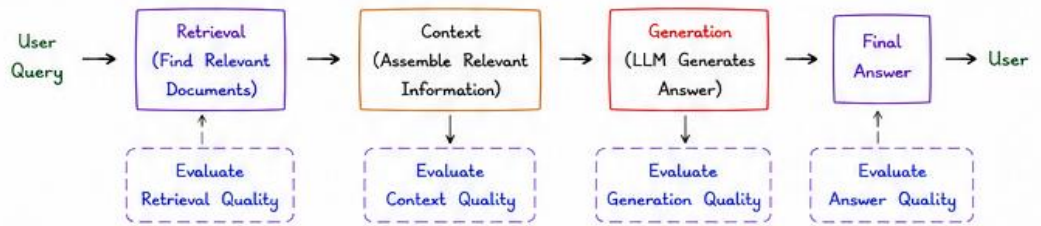
RAG Evaluation

(Retrieval-Augmented Generation Evaluation)

Why Evaluate RAG?

- Ensures the system retrieves the right information.
- Checks if the retrieved context is used effectively by the LLM.
- Measures the quality, accuracy and reliability of the final answer.
- Helps in continuous improvement and regression testing.
- Critical for production readiness and user trust.

RAG Pipeline & Evaluation Points



1. Retrieval Evaluation (Did we retrieve the right docs?)

Information Retrieval Metrics

- Recall@k** : How many relevant docs are retrieved?
- Precision@k** : How many retrieved docs are relevant?
- F1 Score@k** : Harmonic mean of Precision and Recall.
- MRR (Mean Reciprocal Rank)** : Rank of first relevant document.
- NDCG@k (Normalized DCG)** : Considers rank positions of relevant docs.
- Hit Rate@k** : Was at least one relevant doc retrieved?

Definition

$$\text{Precision@k} = \frac{\# \text{ Relevant Retrieved}}{k}$$
$$\text{Recall@k} = \frac{\# \text{ Relevant Retrieved}}{\# \text{ Total Relevant}}$$
$$\text{F1@k} = \frac{2 \times \text{Precision@k} \times \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}}$$
$$\text{MRR} = \frac{1}{\text{Rank of first relevant doc}}$$
$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

2. Context Evaluation (Is the context good enough?)

Context Quality Metrics

- Context Relevance** : How relevant is the context to the query?
- Context Precision** : Proportion of relevant information in context.
- Context Recall** : Does the context cover all necessary info?
- Context Coverage** : Are key aspects/criteria covered?
- Context Redundancy** : How much duplicated or unnecessary info?

How to Measure?

LLM as Judge (e.g., GPT-4) rate context on:

- Relevance (1-5)
- Completeness (1-5)
- Usefulness (1-5)
- Conciseness (1-5)

Or use human evaluation on annotated datasets.

Key Dimensions

- Relevance
- Completeness
- Correctness
- Non-redundancy
- Sufficiency

3. Generation Evaluation (Did the LLM use the context well?)

Generation Quality Metrics

- Faithfulness** : Is the answer consistent with the provided context?
- Context Utilization** : How well did the LLM use the context?
- Helps / Support** : Does the context actually support the answer?
- Hallucination Rate** : Are there statements not in context?
- Answer Relevancy** : Is the answer relevant to the query?

LLM as Judge Prompts

"Given the query, context and answer, rate the following (1-5):"

- Faithfulness
- Relevancy
- Helpfulness
- Hallucination (Higher = worse)

Or use task-specific rubrics & rules.

4. Answer Evaluation (Is the final answer good?)

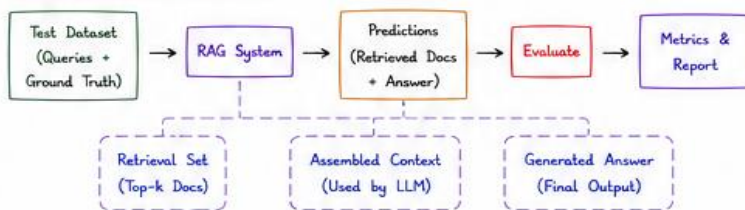
Answer Quality Metrics

- Answer Correctness** : Is the answer factually correct?
- Answer Relevancy** : Does it address the user question?
- Completeness** : Does it answer all parts of the query?
- Clarity / Readability** : Is it easy to understand?
- User Satisfaction** : Overall user happiness with the answer.

Evaluation Methods

- LLM as Judge
- Human Evaluation
- Automatic Metrics (e.g., BLEU, ROUGE, BERTScore)
- User Feedback

Evaluation Framework



End-to-End Evaluation Approaches

Reference-based (with Ground Truth)

- Compare against human-written answers using metrics.
- Needs annotated datasets.
- Metrics: Exact Match, F1, ROUGE, BERTScore, etc.

Reference-free (LLM as Judge)

- Use LLM to judge answer quality.
- Rate on rubrics (Correctness, Relevance, Completeness, etc.)
- Cost-effective for open-domain tasks.

Common Evaluation Datasets

Open-domain QA

- Natural Questions
- TriviaQA
- MS MARCO
- HotpotQA

Domain-specific

- FinanceBench
- HealthSearchQA
- SciFact
- GovReport

Multi-hop / Complex

- 2WikiMultiHopQA
- MuSiQue
- Bamboogle

Instruction / Chat

- RAGAS Evaluation Sets
- ChatDoctor
- LongEval

RAGAS (RAG Assessment Framework)

- Open-source library to evaluate RAG systems.
- Provides metrics for retrieval, context and answer.
- Easy integration with LangChain / LlamaIndex.

Key Metrics in RAGAS



Error Analysis (What to Look For?)

- Irrelevant docs retrieved
- Missing important information
- Contradictory or incorrect info in context
- LLM ignores context
- Hallucinations in answer
- Incomplete or vague answers

What to Monitor in Production?

- Retrieval Success Rate
- Answer Helpfulness (User Feedback)
- Hallucination Alerts
- Latency & Throughput
- Cost per Query
- Drop in metrics over time

Best Practices

- ✓ Create high-quality evaluation dataset.
- ✓ Evaluate each stage separately.
- ✓ Use LLM-as-a-Judge + human eval.
- ✓ Track metrics continuously.
- ✓ Analyze failure cases and iterate.
- ✓ Set thresholds & alerts in production.

Use Cases

- Model / Prompt Comparison
- Chunking & Embedding Tuning
- Retriever / Index Evaluation
- Regression Testing
- A/B Testing
- Production Monitoring

Evaluation Checklist

1. Retrieval

- ☐ Recall@k
- ☐ Precision@k
- ☐ MRR
- ☐ NDCG@k
- ☐ Hit Rate@k

2. Context

- ☐ Context Relevance
- ☐ Context Precision
- ☐ Context Recall
- ☐ Context Coverage
- ☐ Redundancy Check

3. Generation

- ☐ Faithfulness
- ☐ Context Utilization
- ☐ Helpfulness
- ☐ Hallucination Rate
- ☐ Answer Relevancy

4. Answer

- ☐ Correctness
- ☐ Completeness
- ☐ Clarity
- ☐ User Satisfaction

5. System

- ☐ Latency
- ☐ Cost
- ☐ Stability
- ☐ Scalability

Golden Rule

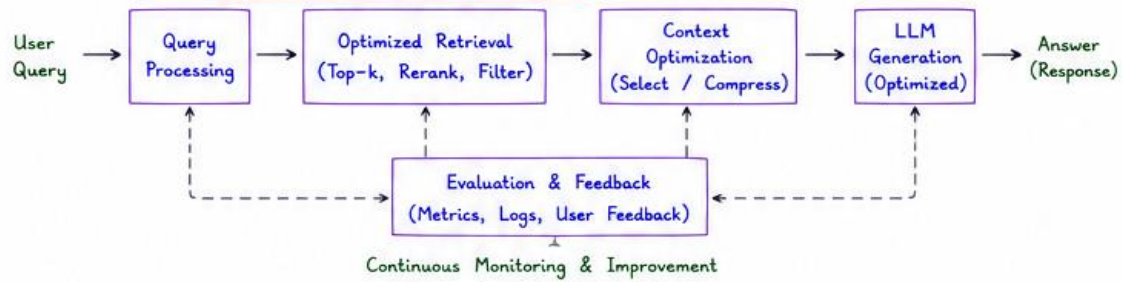
"Evaluate not just the answer, but the entire journey from the query to the final output."

RAG Optimization

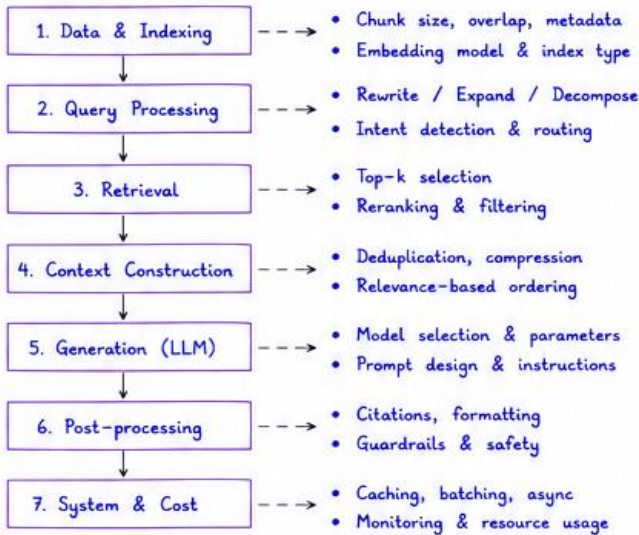
What is RAG Optimization?

- Improves the quality, speed, cost and reliability of RAG systems.
- Optimizes each step: retrieval, context construction, generation and feedback.
- Focuses on relevance, efficiency, cost control and user satisfaction.
- Continuously evaluates and fine-tunes the system.

Optimization Flow (Overview)



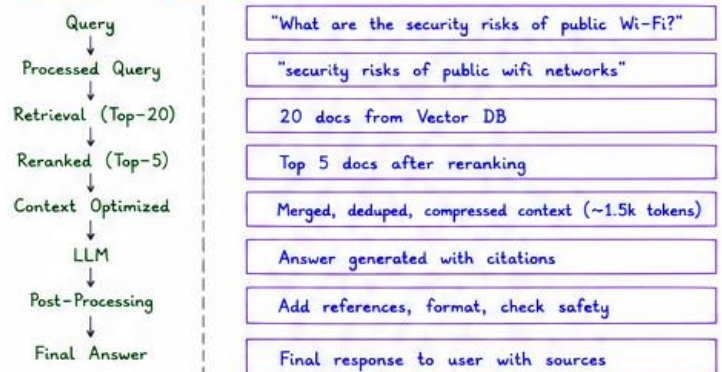
Where to Optimize?



How it Works (Step-by-Step)

- User submits a query.
- Query is processed and optimized (rewrite, expand, route).
- Optimized retrieval fetches top-k relevant documents.
- Reranker/filters select the most relevant and diverse docs.
- Context is optimized (compress, deduplicate, order).
- Optimized prompt is built and sent to LLM.
- LLM generates the answer.
- Post-processing is applied (citations, formatting, safety checks).
- Metrics are collected and system is improved iteratively.

Example (Mini Flow)



Key Optimization Techniques

Retrieval Optimization	Context Optimization	LLM Optimization	System Optimization	Evaluation & Feedback
- Hybrid Search (Dense + Sparse) - Reranking (Cross-Encoder) - Top-k tuning - Metadata filtering - Query decomposition - Dynamic k selection	- Deduplication - Relevance ordering - Compression / Summarization - Sliding window - Max tokens management - Remove low-value info	- Right model for the task - Temperature / Top-p tuning - Prompt engineering - System instructions - Stop sequences - Tool / Function calling when needed	- Caching (Query/Response/Docs) - Batching & Async requests - Parallel retrieval - Efficient embeddings - GPU / Resource utilization - Cost & latency balancing	- Automatic metrics - User feedback - A/B testing - Error analysis - Continuous fine-tuning - Active learning loop

Benefits

- ✓ Higher answer relevance and accuracy
- ✓ Lower latency and better user experience
- ✓ Reduced hallucinations
- ✓ Cost-effective and efficient
- ✓ Scalable and production-ready
- ✓ Continuous improvement over time

Challenges

- ✗ Choosing the right metrics
- ✗ Trade-off between cost and quality
- ✗ Complex pipeline tuning
- ✗ Needs high-quality data & index
- ✗ Evaluation is not always ground-truth based
- ✗ Keeping system updated and monitored

Optimization Areas vs Impact

Area	Focus	Impact	Typical Techniques
Retrieval	Get the right docs	★★★★★	Hybrid search, Rerank, Filters
Context	Provide the right context	★★★★☆	Compress, Dedup, Reorder
LLM	Generate better answers	★★★★☆	Prompting, Params, Model choice
System	Run faster & cheaper	★★★★☆	Caching, Batching, Monitoring
Feedback	Improve over time	★★★★★	Metrics, Logs, User Feedback

Best Practices

- Start simple, measure, then optimize.
- Focus first on retrieval and context.
- Use evaluation metrics on real data.
- Automate feedback collection.
- Monitor latency, cost and quality.
- Continuously iterate and improve.



Use Cases

- Enterprise Knowledge Assistants
- Customer Support Chatbots
- Research & Academic Search
- Legal / Compliance Systems
- Healthcare Information Systems
- Technical Documentation Search

Key Metrics to Track

- Answer Relevance (Recall@k)
- Accuracy (Faithfulness)
- Hallucination Rate
- User Satisfaction
- Latency (P50 / P95)
- Cost per 1K queries
- Tokens Used
- Error / Fail Rate
- Coverage / Completeness
- Click-through / Usage

Production RAG

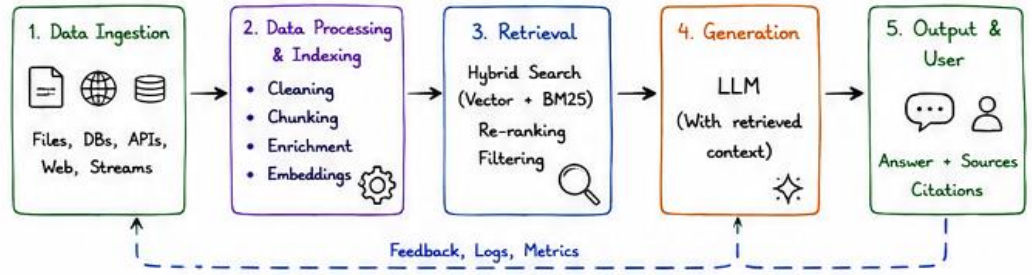
(Building Reliable, Scalable & Secure RAG Systems for Real-world)

What is Production RAG?

- Takes RAG from prototype to production-ready systems.
- Focuses on reliability, scalability, security, monitoring and cost.
- Ensures high quality answers with low latency and high availability.
- Handles real-world challenges like failures, data freshness, and users at scale.



Production RAG Pipeline (High Level)



Core Components

Data Layer	Reliable data sources, versioning, access control, freshness.
Index Layer	Vector DB / Search Engine (eg. pgvector, Pinecone, Weaviate, Elasticsearch)
Retrieval Layer	Query routing, hybrid search, re-ranking, filters, query expansion.
Generation Layer	LLM with prompt templates, context management, guardrails.
Ops & Governance	Security, monitoring, evaluation, logging, cost & scalability.

Key Production Concerns

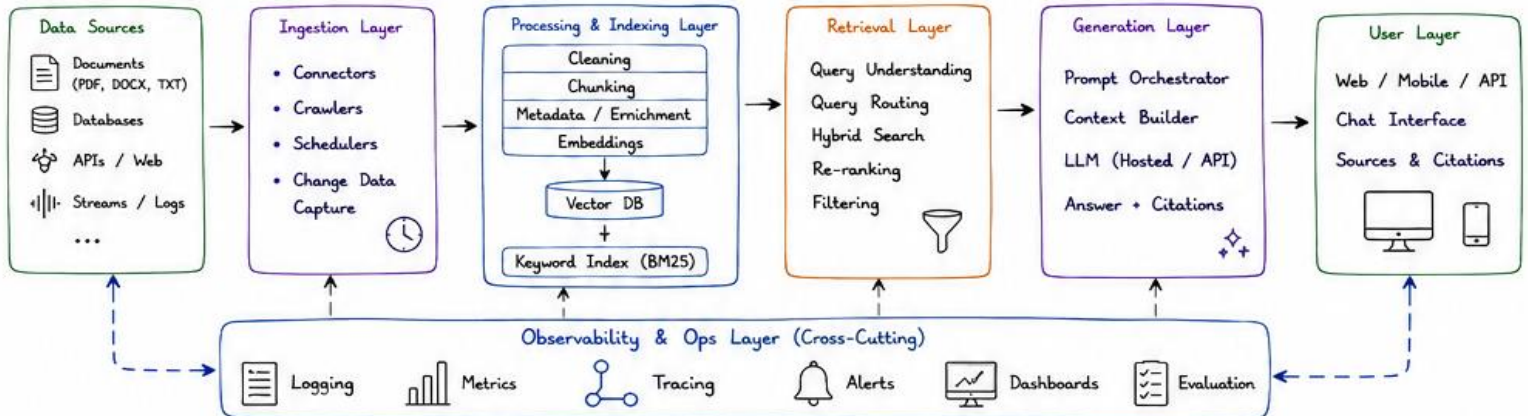
- ✓ Accuracy & Relevance (reduce hallucinations)
- ✓ Latency & Throughput (meet SLAs)
- ✓ Scalability (handle more users & data)
- ✓ Reliability & Availability (no downtime)
- ✓ Data Freshness (up-to-date information)
- ✓ Security & Privacy (protect data)
- ✓ Cost Optimization (inference & storage)
- ✓ Observability (logs, metrics, tracing)
- ✓ Evaluation & Continuous Improvement



Production Challenges & Solutions

Challenges	Solutions
Irrelevant / low quality retrieval	Hybrid search, re-ranking, better chunking, metadata filters
Hallucinations	Better context, citations, guardrails, answer verification
High latency	Query caching, compact prompts, parallel retrieval, stream response
Data changes frequently	Incremental indexing, versioning, freshness checks
High cost	Model routing, small models for simple queries, caching
Security risks	Access control, PII masking, encryption, audit logs
Failures & outages	Retries, fallbacks, circuit breakers, monitoring & alerts

Production RAG Architecture (Reference)



Best Practices

- ✓ Use hybrid search + re-ranking
- ✓ Design good chunking strategies
- ✓ Add rich metadata & filters
- ✓ Use citations / sources in answers
- ✓ Implement caching (query, embedding, response)
- ✓ Monitor everything (latency, errors, quality)
- ✓ Evaluate regularly & iterate
- ✓ Secure data & control access
- ✓ Plan for scale (horizontal scaling)



Evaluation in Production

- Offline Evaluation: Recall@k, Precision@k, MRR, NDCG, Faithfulness, Answer Relevancy
- Online Evaluation: User Feedback, Thumbs up/down, Session success rate
- A/B Testing: Different retrieval / prompts / models
- Human-in-the-loop: Review low confidence or low quality answers



Monitoring Metrics

- Retrieval: Recall@k, Hit Rate, Latency
- Generation: Latency, Tokens/sec, Cost
- Quality: Faithfulness, Relevancy, Correctness
- System: QPS, Error Rate, Uptime
- Business: User Satisfaction, Task Success



Security & Compliance

- Role-based access control
- PII detection & masking
- Data encryption (at rest/in transit)
- Audit logs & access monitoring
- Comply with GDPR, SOC2, etc.



RAG Deployment Options



LLM Deployment Strategies

- Use hosted LLMs (OpenAI, Anthropic, etc.)
- Use open-source LLMs (Mixtral, Llama, etc.)
- Use small models for routing / filtering
- Distill / fine-tune for domain tasks
- Fallback models for cost & reliability



Cost Optimization Tips

- Cache embeddings & results
- Use smaller models when possible
- Limit context window smartly
- Batch requests
- Monitor & cap usage



Common Failure Points

- ✗ Poor chunking & indexing
- ✗ Wrong retrieval (irrelevant docs)
- ✗ Outdated or stale data
- ✗ Prompt not aligned to task
- ✗ Long context → high latency/cost
- ✗ No monitoring or evaluation



Checklist for Production RAG

- ☐ Data ingestion reliable & automated
- ☐ Data processing & indexing optimized
- ☐ Hybrid search + re-ranking in place
- ☐ Prompt templates & guardrails ready
- ☐ Monitoring, alerts & dashboards set
- ☐ Evaluation pipeline defined
- ☐ Security & access controls verified
- ☐ Cost & scaling tested
- ☐ User feedback loop enabled



Tools & Technologies

- Ingestion: Airbyte, NiFi, Kafka, Crawlers
- Processing: LangChain, LlamaIndex, Haystack
- Vector DB: Pinecone, Weaviate, Qdrant, pgvector
- Search: Elasticsearch, OpenSearch
- Orchestration: LangChain, LlamaIndex
- Monitoring: Prometheus, Grafana, LangSmith
- Deployment: Docker, Kubernetes, Terraform



Use Cases

- ★ Enterprise Knowledge Search
- ★ Customer Support Chatbots
- ★ Document Q&A & Summarization
- ★ Legal / Compliance Research
- ★ Financial & Market Intelligence
- ★ Healthcare & Medical Assistants
- ★ Education & Learning Assistants

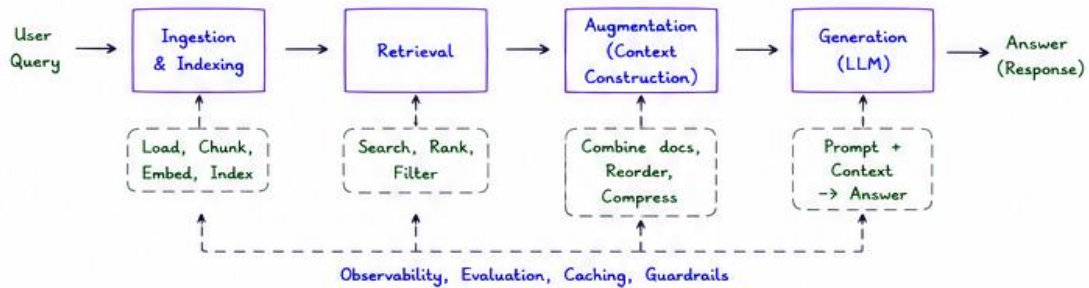


RAG Frameworks

What are RAG Frameworks?

- RAG frameworks provide tools and abstractions to build, manage and scale Retrieval Augmented Generation applications easily.
- They handle retrieval, indexing, chunking, orchestration, memory, agents, evaluation and integrations.
- Save development time and follow best practices out-of-the-box.

RAG Pipeline (Generic Flow)



Popular RAG Frameworks

Framework	Language	Key Strengths	Best For
LangChain	Python / JS	Most popular, flexible, huge ecosystem, integrations, agents, chains, tools	General purpose RAG, prototyping to production
LlamaIndex (Formerly GPT Index)	Python	Data connectors, indexing, query engines, easy to work with private data	RAG over private docs, structured & unstructured data
Haystack (deepset)	Python	Production ready, fast, scalable, built-in pipelines, retrievers, rankers	Enterprise RAG, scalable search applications
Microsoft Semantic Kernel	Python/ C# / JS	Orchestration, plugins, memory, skills, multi-agent support	Enterprise apps, .NET ecosystem, agents
AutoGen	Python	Multi-agent conversations, automation, tool use, planning	Multi-agent RAG systems, autonomous workflows
RAGFlow	Python / JS	End-to-end RAG platform, UI, chunking, parsing, retrieval, LLM orchestration	Quick setup, full RAG app with UI
LanceDB + LlamaIndex	Python	High performance vector DB + easy RAG tooling	High scale vector search + RAG
Milvus / Weaviate + LangChain	Python	Powerful vector DBs with LangChain integration	Large scale, distributed RAG systems
CrewAI	Python	Multi-agent orchestration with roles and tasks	Agentic RAG, role-based workflows

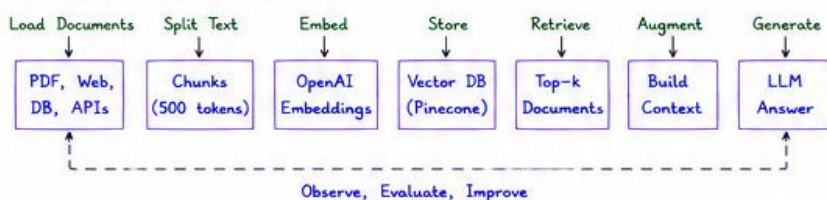
How RAG Frameworks Help

- ✓ Simplify ingestion, chunking and indexing
- ✓ Provide retrievers, re-rankers and filters
- ✓ Build context and prompts automatically
- ✓ Integrate with LLMs and tools easily
- ✓ Support memory, agents and workflows
- ✓ Offer observability, tracing and evaluation
- ✓ Scale from prototype to production

Key Features Across Frameworks

- Document loaders & connectors (Web, DB, APIs, Files)
- Text splitting (Recursive, Semantic, Token based)
- Embeddings (OpenAI, HuggingFace, Cohere, etc.)
- Vector stores (FAISS, Chroma, Pinecone, Milvus, Weaviate, etc.)
- Retrieval (Dense, Sparse, Hybrid, MMR, Rerankers)
- Context construction (Compression, Reordering, Fusion)
- LLM integration (OpenAI, Anthropic, Llama, Mistral, etc.)
- Memory (Buffer, Summary, Entity, Vector, Episodic)
- Agents & Tools (Planning, Function calling, Actions)
- Evaluation & Monitoring (Metrics, Tracing, Logs)

Example: RAG with LangChain (High Level)



Choosing the Right Framework

Need / Scenario	Recommended Frameworks
Quick prototype	LangChain, LlamaIndex, RAGFlow
Private data over many docs	LlamaIndex, Haystack
Enterprise production system	Haystack, Microsoft Semantic Kernel
Multi-agent RAG	AutoGen, CrewAI, LangChain + Agents
.NET / Azure ecosystem	Microsoft Semantic Kernel
Built-in UI & Full Platform	RAGFlow
High scale vector search	LanceDB, Milvus, Weaviate + LangChain

Comparison Snapshot

Framework	Ease of Use	Flexibility	Production Readiness	Ecosystem	Agents Support	UI / Platform
LangChain	★★★★☆	★★★★★	★★★☆☆	★★★★★	★★★★★	No
LlamaIndex	★★★★☆	★★★★★	★★★☆☆	★★★★★	★★★☆☆	No
Haystack	★★★★☆	★★★★★	★★★★★	★★★★★	★★★☆☆	No
Semantic Kernel	★★★★☆	★★★★★	★★★★★	★★★★★	★★★★★	No
AutoGen	★★★★☆	★★★★★	★★★★★	★★★★★	★★★★★	No
RAGFlow	★★★★☆	★★★★★	★★★★★	★★★★☆	★★★★★	Yes

Typical Stack (Example with LangChain)

Frontend	Streamlit / React / Next.js
Backend	FastAPI / Flask / Node.js
RAG Framework	LangChain / LlamaIndex
Embeddings	OpenAI / HuggingFace / Cohere
Vector DB	Pinecone / Chroma / Milvus / Weaviate
LLM	OpenAI GPT / Claude / Llama / Mistral
Monitoring	LangSmith / Arize / Weights & Biases

Benefits

- ✓ Faster development
- ✓ Fewer boilerplate code
- ✓ Best practices built-in
- ✓ Easy integrations
- ✓ Scalable and maintainable
- ✓ Active communities & support

Challenges

- ✗ Learning curve for advanced features
- ✗ Vendor lock-in for some services
- ✗ Cost of APIs & vector DBs
- ✗ Performance tuning still required
- ✗ Security & data privacy management

Best Practices

- Choose the right chunking strategy
- Use hybrid retrieval + reranking
- Compress and order context well
- Add citations and guardrails
- Evaluate with real user queries
- Monitor, log and improve continuously

Use Cases

- Chatbots over private documents
- Knowledge assistants
- Research & summarization
- Legal & compliance search
- Technical support bots
- Enterprise knowledge search

Key Metrics

- Answer Relevance (Recall@k)
- Faithfulness
- Correctness (Exact / F1)
- Hallucination Rate
- Latency (P50 / P95)
- Cost per query

Where to Find & Learn



LangChain Docs
python.langchain.com



LlamaIndex Docs
docs.llamaindex.ai



Haystack Docs
docs.haystack.deepset.ai



Microsoft Semantic Kernel
aka.ms/semantic-kernel



RAGFlow Docs
ragflow.io/docs



Awesome RAG
github.com/kyrolabs/awesome-rag

- ✓ ALL CONCEPTS
- ✓ ALL VARIANTS
- ✓ ALL EXPLAINED!



COMPLETE HANDWRITTEN NOTES

on **RAG** (RETRIEVAL-AUGMENTED GENERATION)

FROM BASICS
TO ADVANCED
TO PRODUCTION!



"RAG combines the power of retrieval and generation to give accurate, reliable & up-to-date answers." ★

What is RAG?



RAG (Retrieval-Augmented Generation) is an AI framework that enhances LLMs by retrieving relevant information from external knowledge sources and using it to generate accurate, context-aware responses.

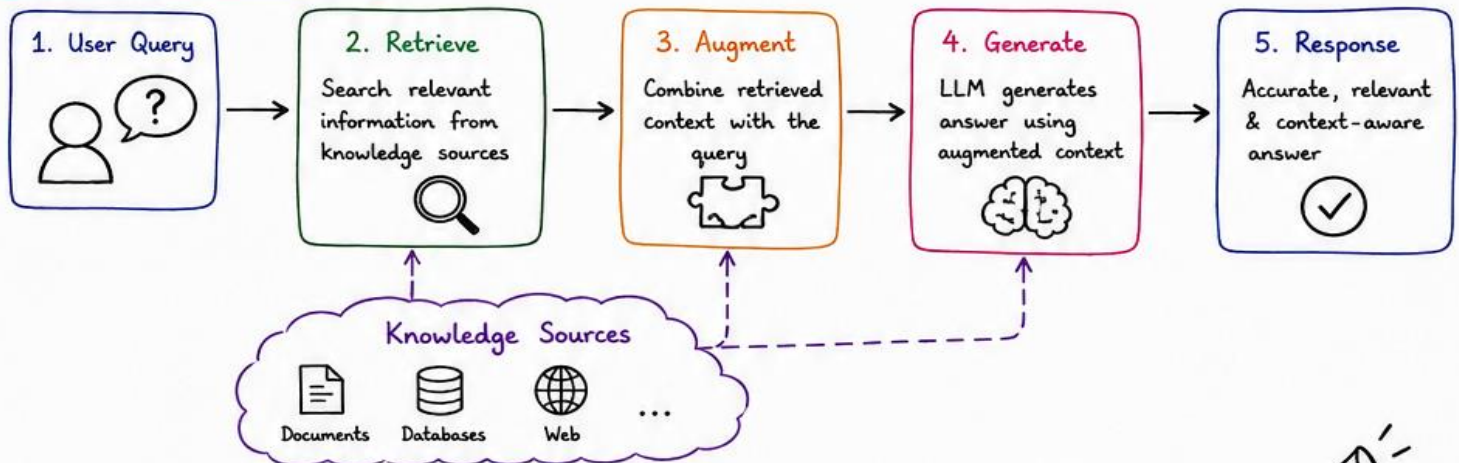
Why Use RAG?



- ✓ Provides up-to-date information
- ✓ Reduces hallucinations
- ✓ Grounds answers in trusted sources
- ✓ Handles private / domain-specific data
- ✓ Improves accuracy & reliability
- ✓ Transparent & explainable



How Does RAG Work?



Key Components of RAG

- 1. Data / Knowledge Sources**
Documents, PDFs, Websites, DBs, APIs, etc.
- 2. Indexing**
Chunking, Embeddings, Metadata, Vector Store / Index
- 3. Retrieval**
Search relevant chunks using similarity / keywords / hybrid search
- 4. Augmentation**
Combine retrieved context with the user query
- 5. Generation**
LLM generates final answer using the augmented context

Benefits of RAG

- ★ **Up-to-date Knowledge**
Access latest information.
- ★ **More Accurate Answers**
Reduces hallucinations.
- ★ **Domain Specific**
Works on private / custom data.
- ★ **Cost Effective**
Smaller LLMs can perform better with retrieved context.
- ★ **Explainable**
Sources can be cited.
- ★ **Scalable**
Easily extendable to large knowledge bases.

Applications

- Enterprise Search
- Customer Support
- Chatbots & Assistants
- Document Q&A
- Research & Academia
- Legal & Compliance
- Healthcare
- Finance & Reports
- Education & E-learning
- And many more... ★



SAVE THIS
POST

COMPLETE GUIDE COVERING
BASICS | ARCHITECTURES | VARIANTS | EVALUATION | PRODUCTION
BEST PRACTICES | TOOLS | USE CASES & MORE!

STAY TUNED!
ALL SLIDES
COMING UP



FOLLOW FOR MORE AI / ML / GENAI CONTENT